Design and Analysis of Computer Algorithms

01:198:344

Spring 2026, Rutgers

Instructor: Surya Teja Gavva

Borůvka's Algorithm for Minimum Spanning Tree (MST)

Borůvka's algorithm finds the MST by repeatedly adding the cheapest edge for each component. The algorithm works by merging components until only one spanning tree remains.

Algorithm Steps:

1. **Initialize**: Start with each vertex in its own component, creating an initial forest F of isolated nodes.
2. **Main Loop**:
   - For each component, find the minimum-weight edge that connects it to a different component.
   - Add these cheapest edges to F, merging components.
   - Update the component count; if F is a single component, the MST is complete.

Borůvka's Algorithm Complexity and Efficiency

Efficient Edge Selection:

- Subroutine `AddAllSafeEdges` identifies the minimum edge for each component in $O(E)$ time, where E is the number of edges.
- Connected component counting runs in $O(V)$, where V is the number of vertices.

Time Complexity:

- Each iteration reduces component count by at least half, so the main loop runs $O(\log V)$ times.
- Each iteration takes $O(E)$, leading to an overall time complexity of $O(E \log V)$.

Applications: Borůvka's algorithm is efficient and well-suited for parallel processing, as minimum edge selection can be done independently for each component.

Optimize Water Distribution in a Village

Problem Statement:

- You are given n houses in a village, each needing water supply.
- Water can be provided in two ways:
  1. Build individual wells for each house, with specified costs.
  2. Connect houses with pipes, with specified costs for each connection.
- Goal: Minimize the total cost to supply water to all houses.

Input:

- `wells` array where `wells[i]` is the cost to build a well for house i.
- `pipes` array where each element $[u, v, cost]$ represents a pipe between houses u and v with a given cost.

Optimize Water Distribution in a Village

Solution:

- Treat each house as a node in a graph.
- Introduce a virtual node connected to each house with an edge (well) weighted by the cost of building a well.
- Add all pipes as edges between houses with their respective costs.
- Use an MST algorithm (e.g., Kruskal's or Prim's) to find the minimum cost to connect all nodes, including the virtual node.

Complexity:

- Graph building: $O(V + E)$, where E is the number of pipes.
- MST computation (Kruskal's or Prim's): $O(E \log E)$.

Rebuild the City

Problem: The city needs to be reconnected with minimum repair costs. Each road has a repair cost, and the goal is to fully reconnect the city at the lowest total expense.

Objective: Minimize the total cost of repairs to keep the city fully connected.

Approach: - Apply Kruskal's or Prim's algorithm to find the Minimum Spanning Tree (MST). - Sort roads by cost (for Kruskal) or use a priority queue (for Prim) to select minimum-cost roads. - Sum the costs of the selected roads to obtain the final answer.

Complexity: $O(E \log E)$, where E is the number of roads.

The Earliest Moment When Everyone Becomes Friends

Determine the earliest time when everyone in a network is connected, given friendship logs `[timestamp, personA, personB]`.

Approach:

1. Sort logs by timestamp.
2. Use Union-Find to track connections between individuals.
3. For each log, `union` the two friends. If only one connected component remains, record the timestamp and stop.

Complexity: Sorting+ Union-Find $O(E \log E)$

Single-Source Shortest Path Problem

- **Problem Definition**: Given a graph $G = (V, E)$ with weights $w(e) > 0$ on each edge $e \in E$ and a source vertex s, find the shortest path from s to each vertex $v \in V$.
- **Goal**: Minimize path weight $w(P) = \sum_{e \in P} w(e)$ for paths from s to v.
- Applications include shortest route mapping, network latency optimization, and more.

Single-Source Shortest Path Problem

- **Problem Definition**: Given a graph $G = (V, E)$ with weights $w(e) > 0$ on each edge $e \in E$ and a source vertex s, find the shortest path from s to each vertex $v \in V$.
- **Goal**: Minimize path weight $w(P) = \sum_{e \in P} w(e)$ for paths from s to v.
- Applications include shortest route mapping, network latency optimization, and more.

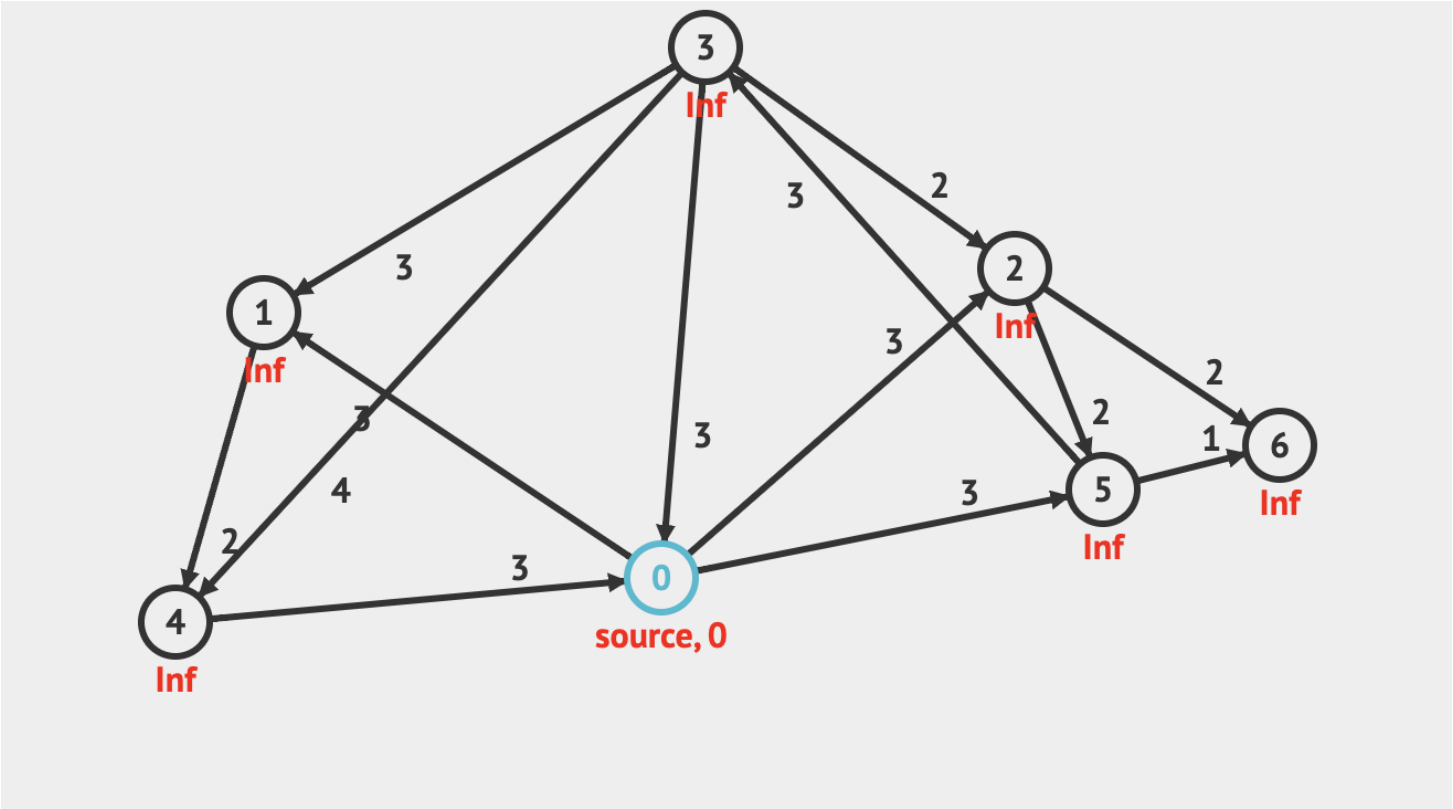
Dijkstra's Algorithm: Overview

- **Purpose**: Computes shortest paths from source s in graphs with non-negative edge weights.
- Similar to Prim's MST algorithm but tailored for shortest paths.
- **Initialization**:
  - Set $d[s] = 0$; $d[v] = +\infty$ for $v \neq s$.
  - Mark all vertices as unvisited.

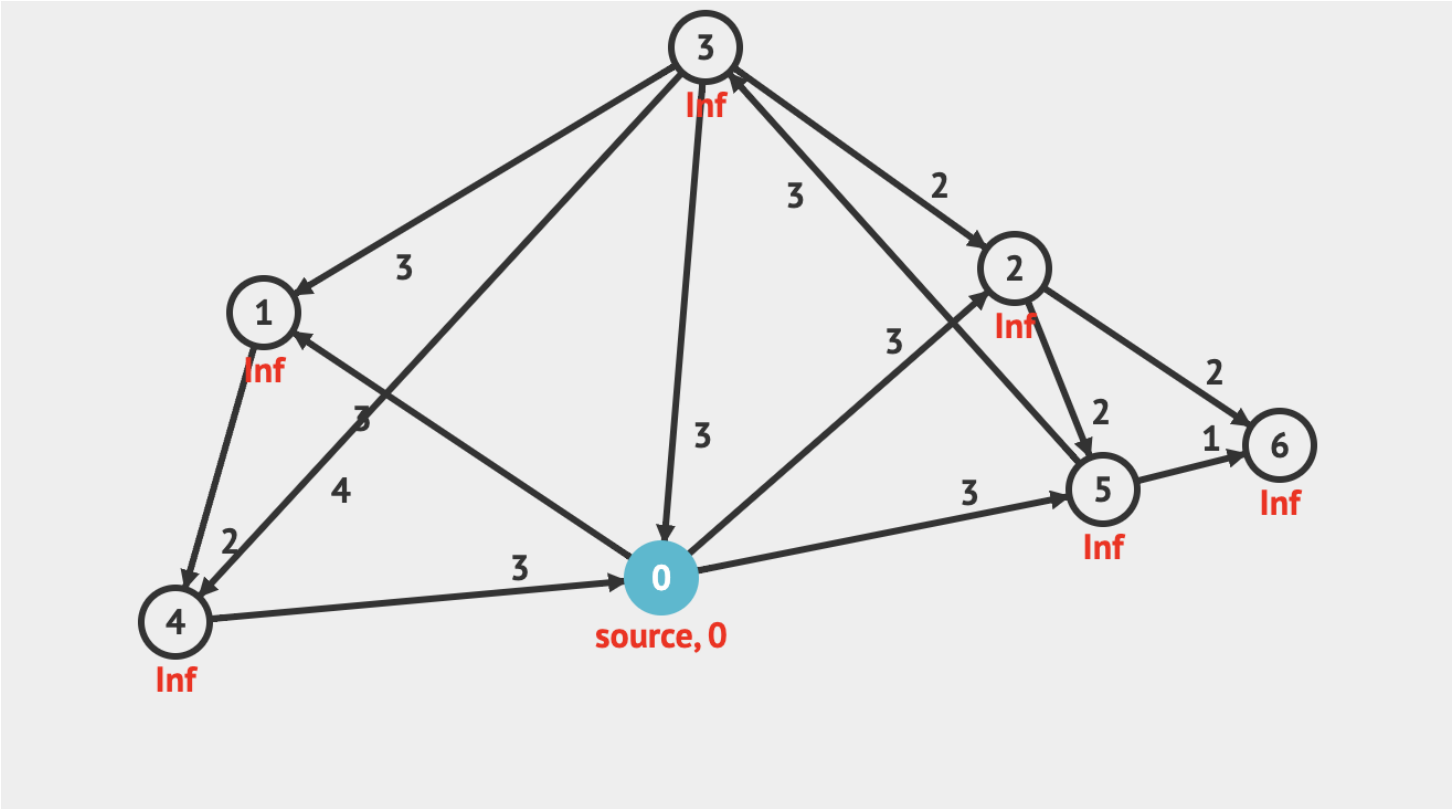
Dijkstra's Algorithm: Steps

1. While there are unvisited nodes:
   - Select the unvisited vertex u with the smallest $d[u]$.
   - For each neighbor v of u, if $d[v] > d[u] + w(u, v)$:
     - Update $d[v] = d[u] + w(u, v)$.
   - Mark u as visited.
2. Repeat until all vertices are visited, then $d[v]$ reflects shortest paths.

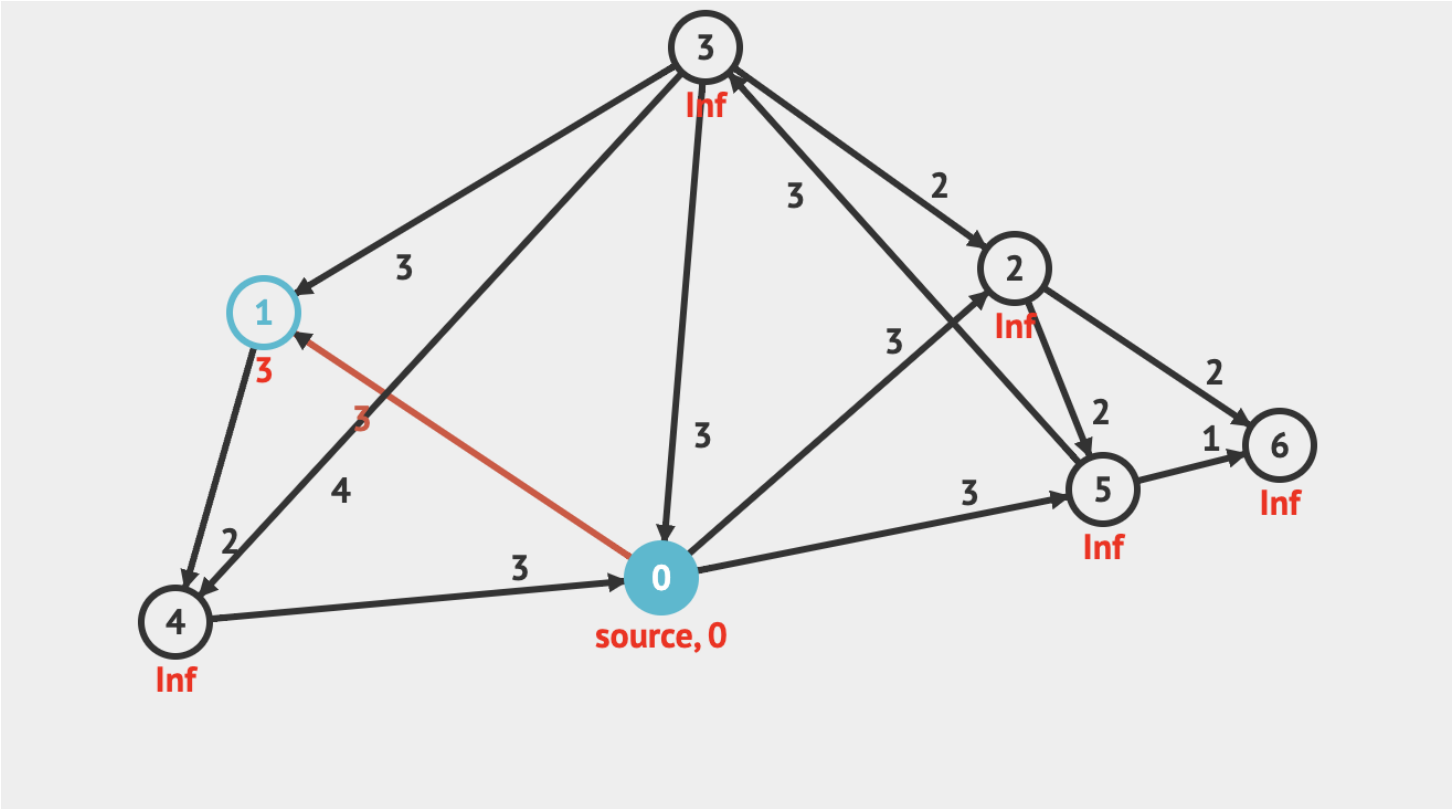
Dijkstra's Algorithm Example



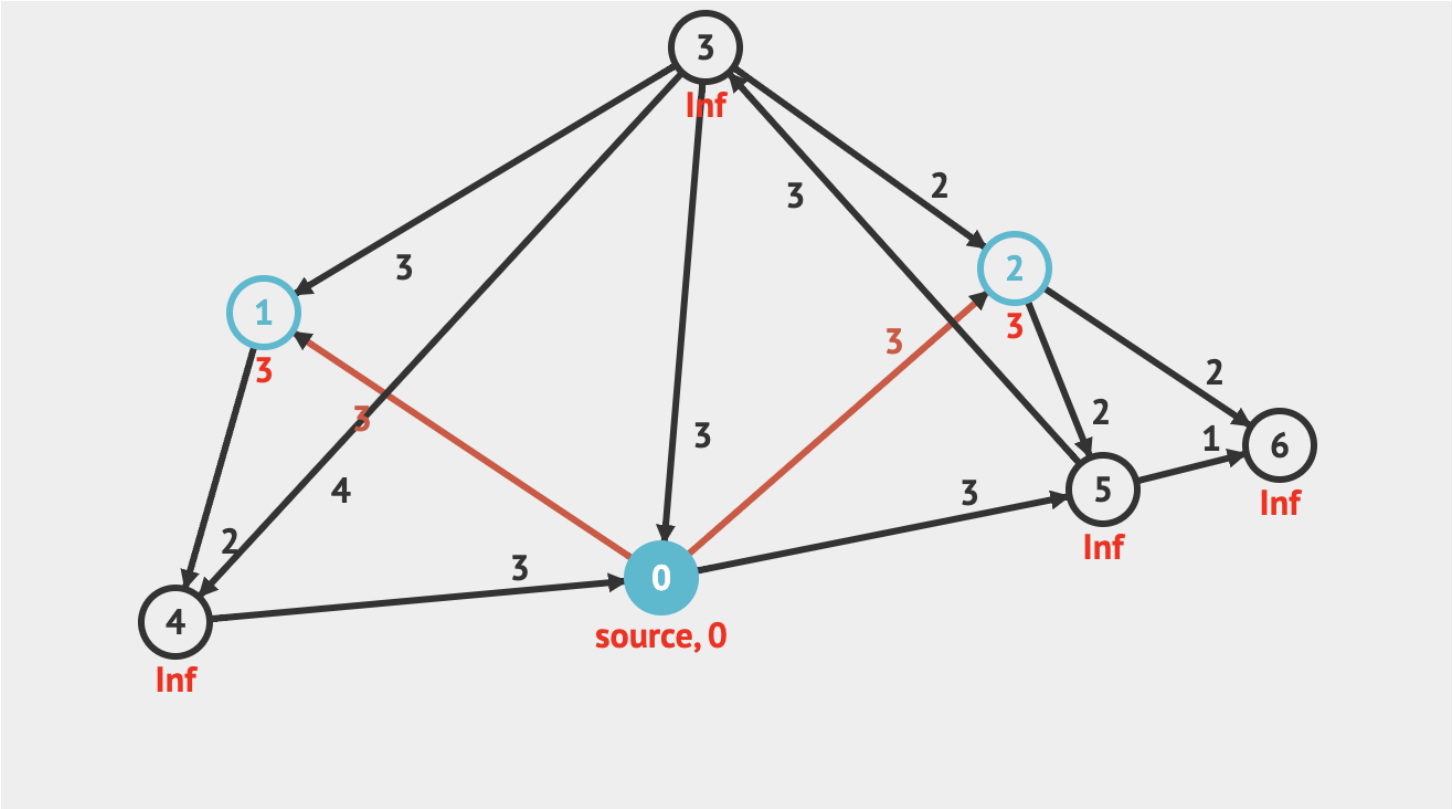
Dijkstra's Algorithm Example



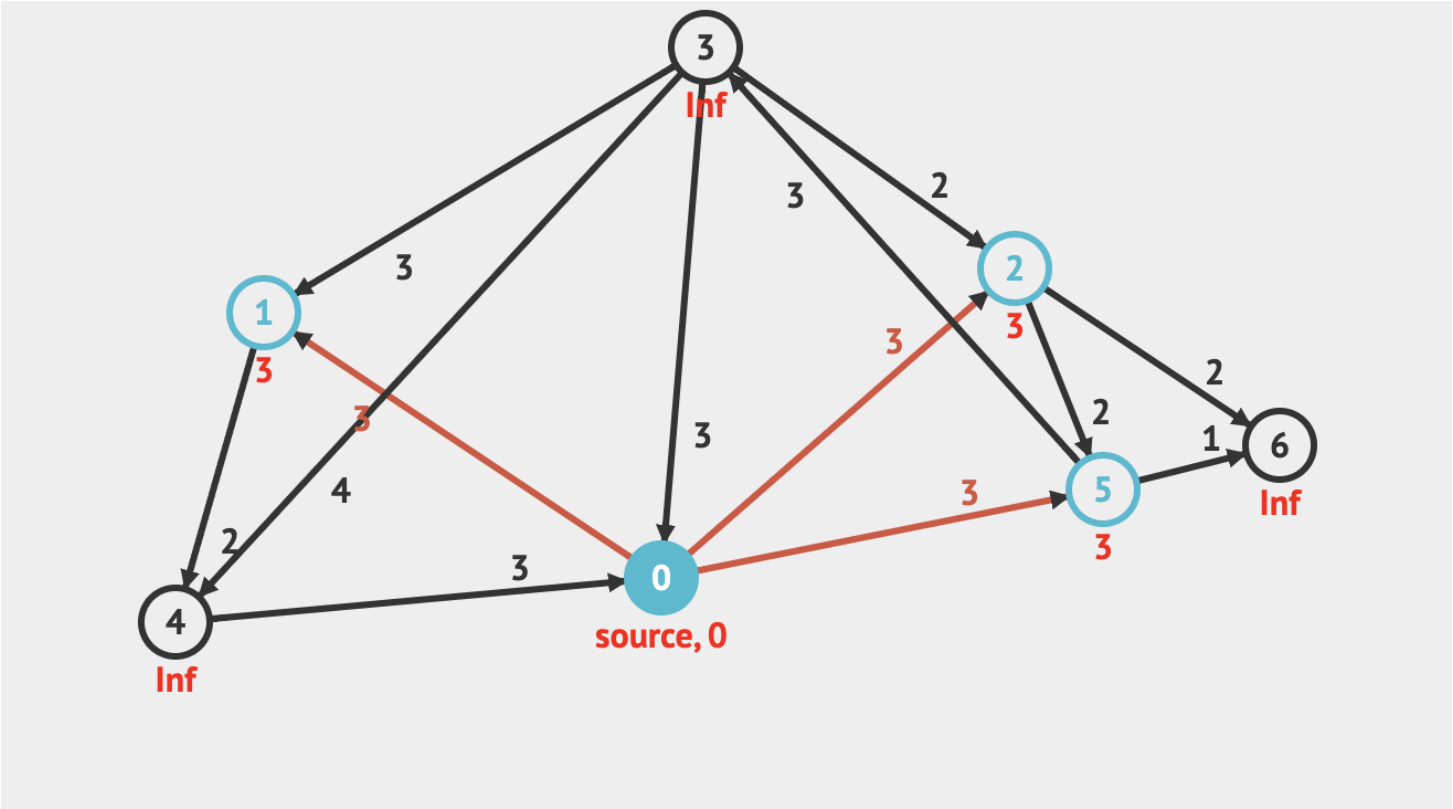
Dijkstra's Algorithm Example



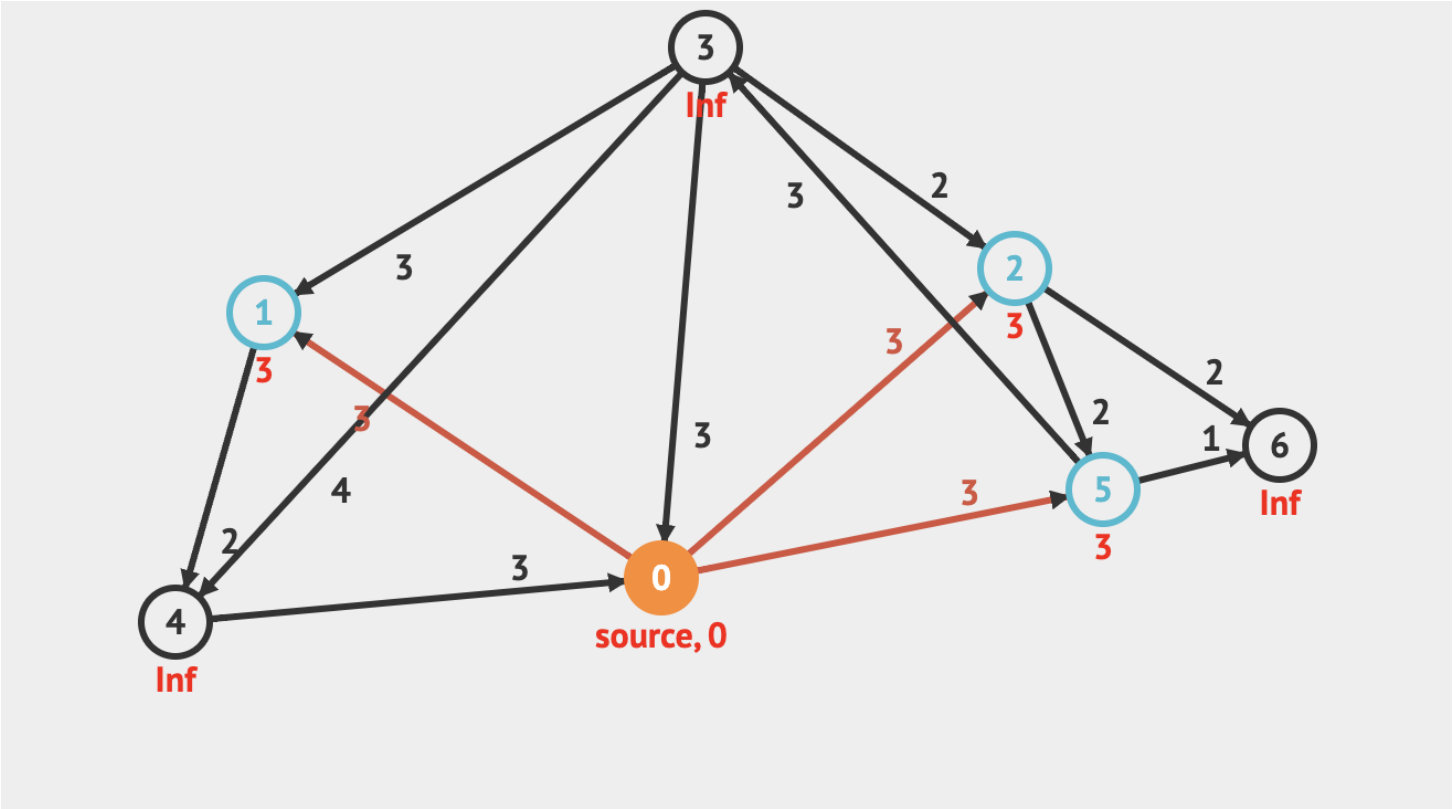
Dijkstra's Algorithm Example



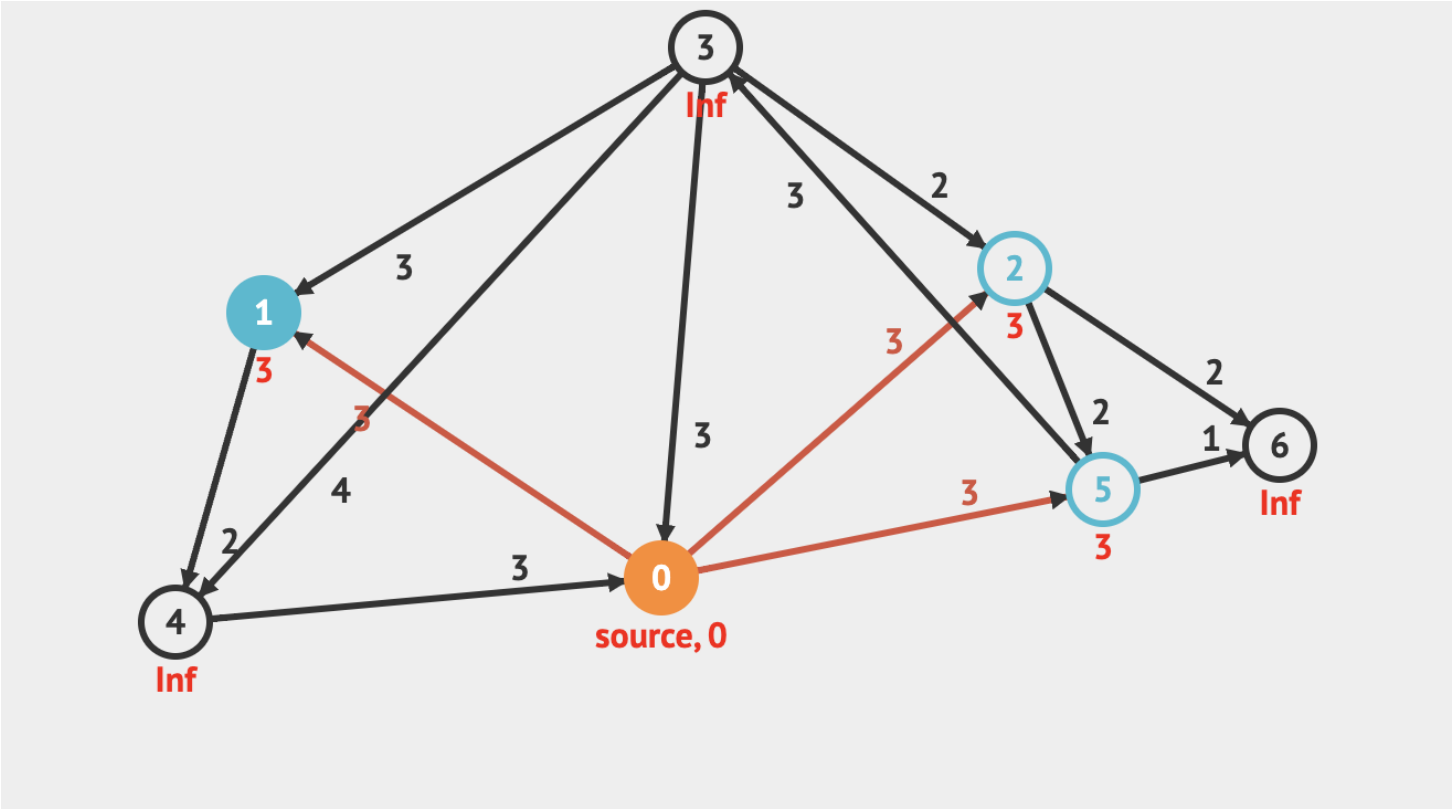
Dijkstra's Algorithm Example



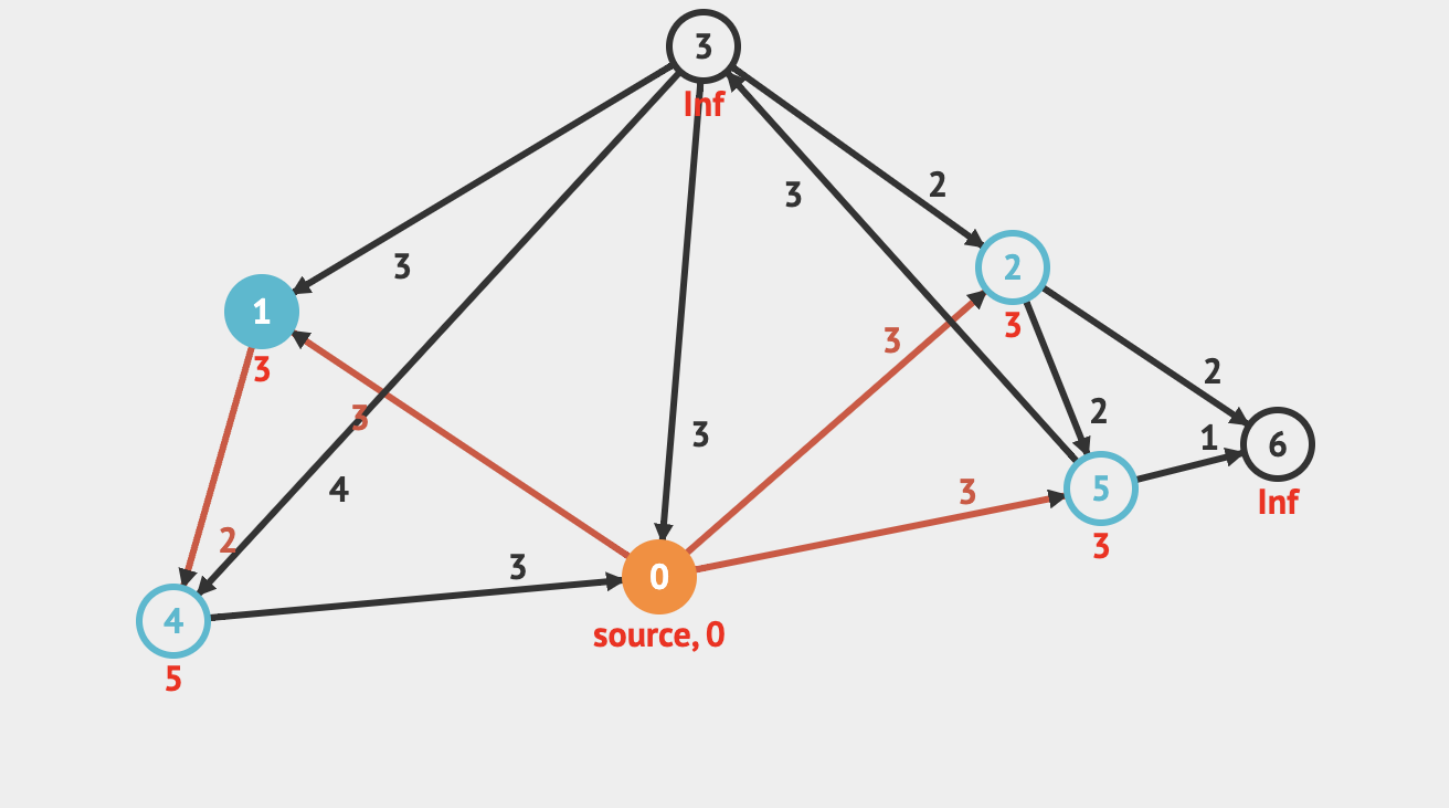
Dijkstra's Algorithm Example



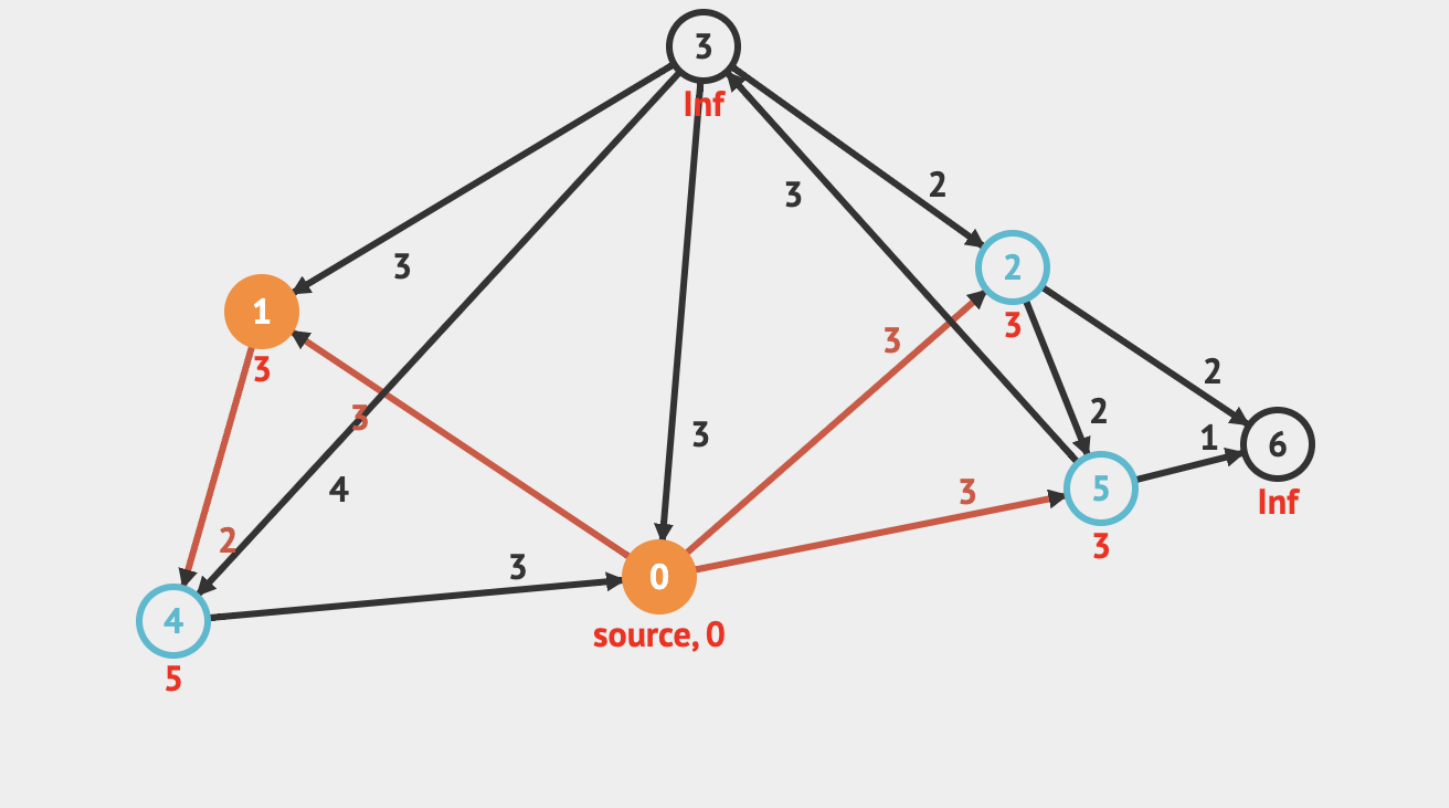
Dijkstra's Algorithm Example



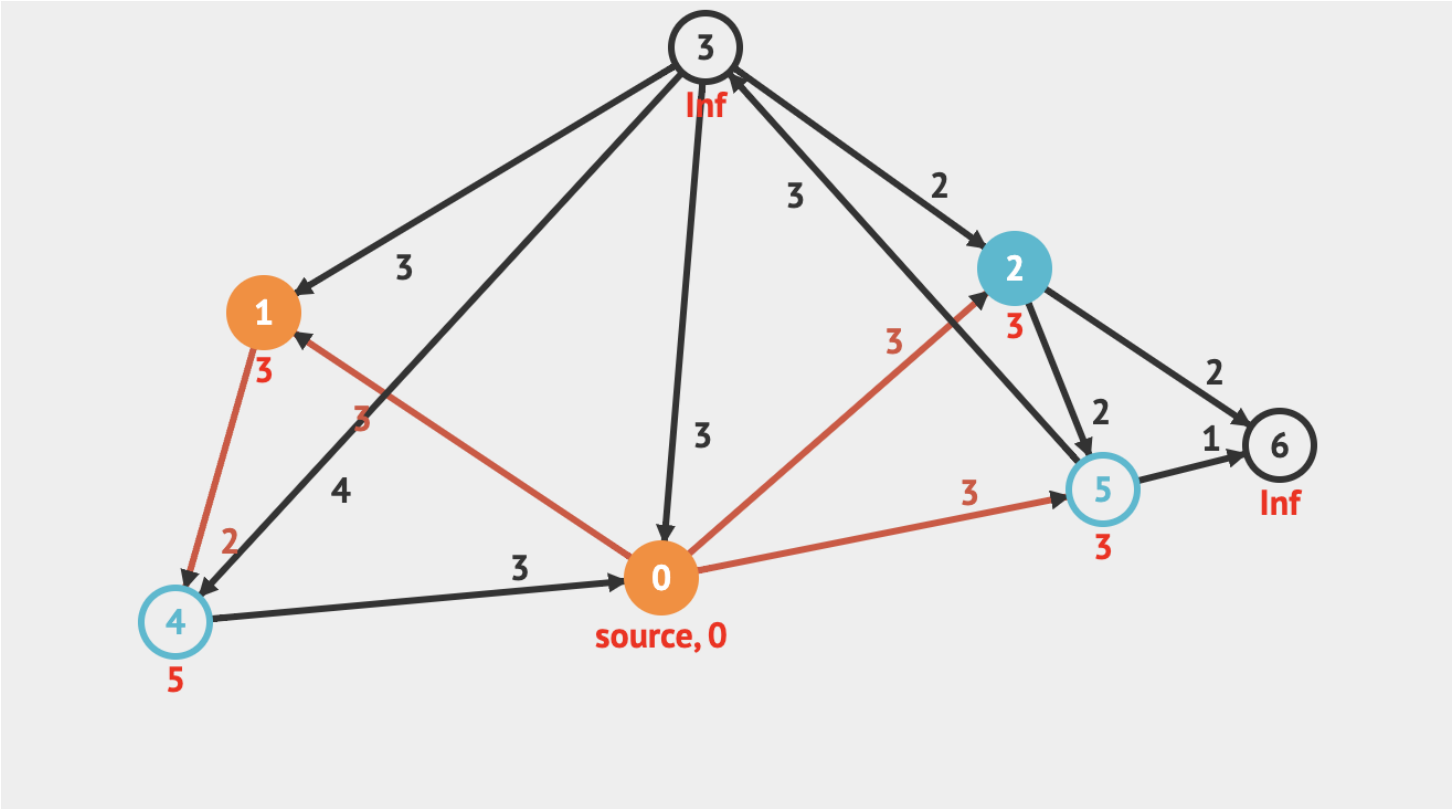
Dijkstra's Algorithm Example



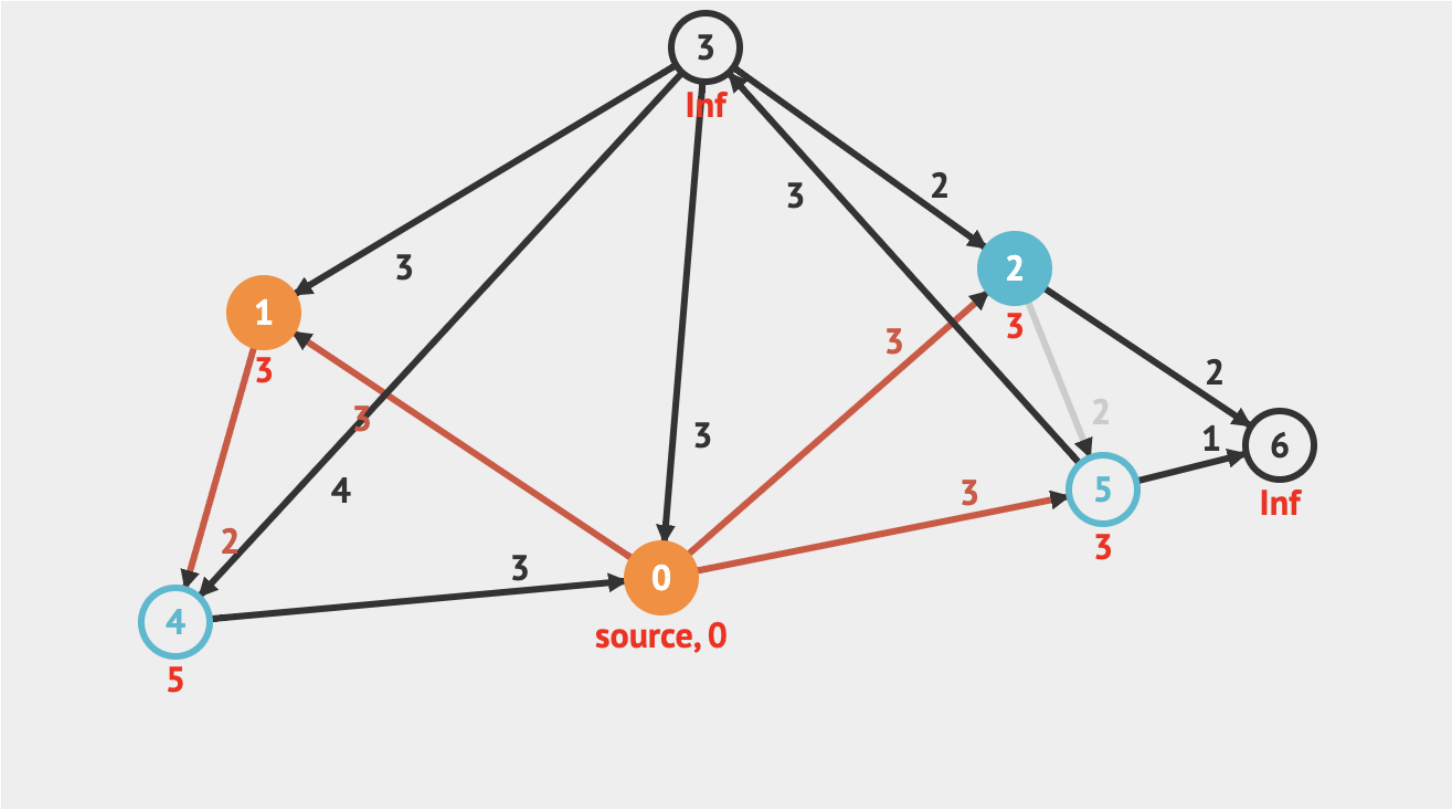
Dijkstra's Algorithm Example



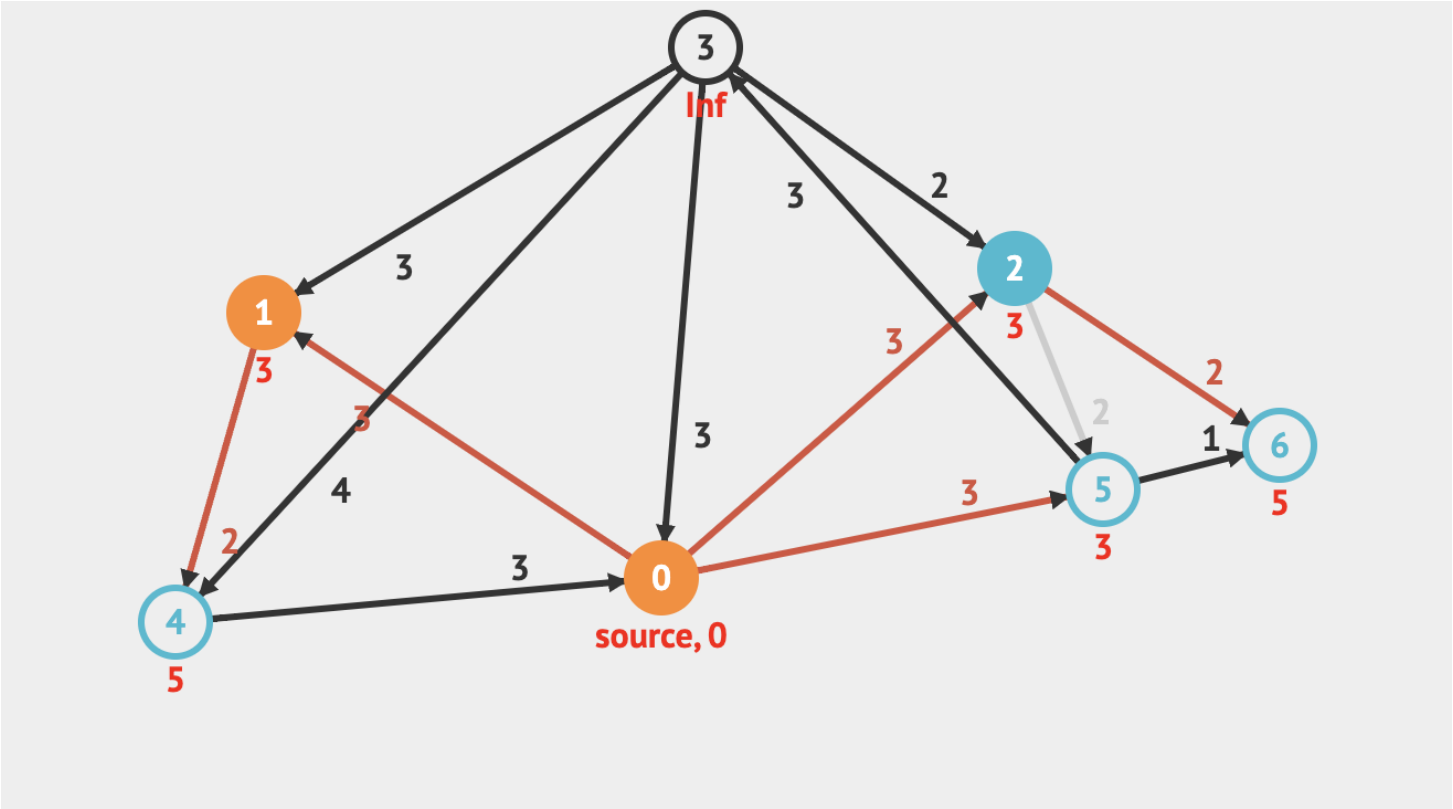
Dijkstra's Algorithm Example



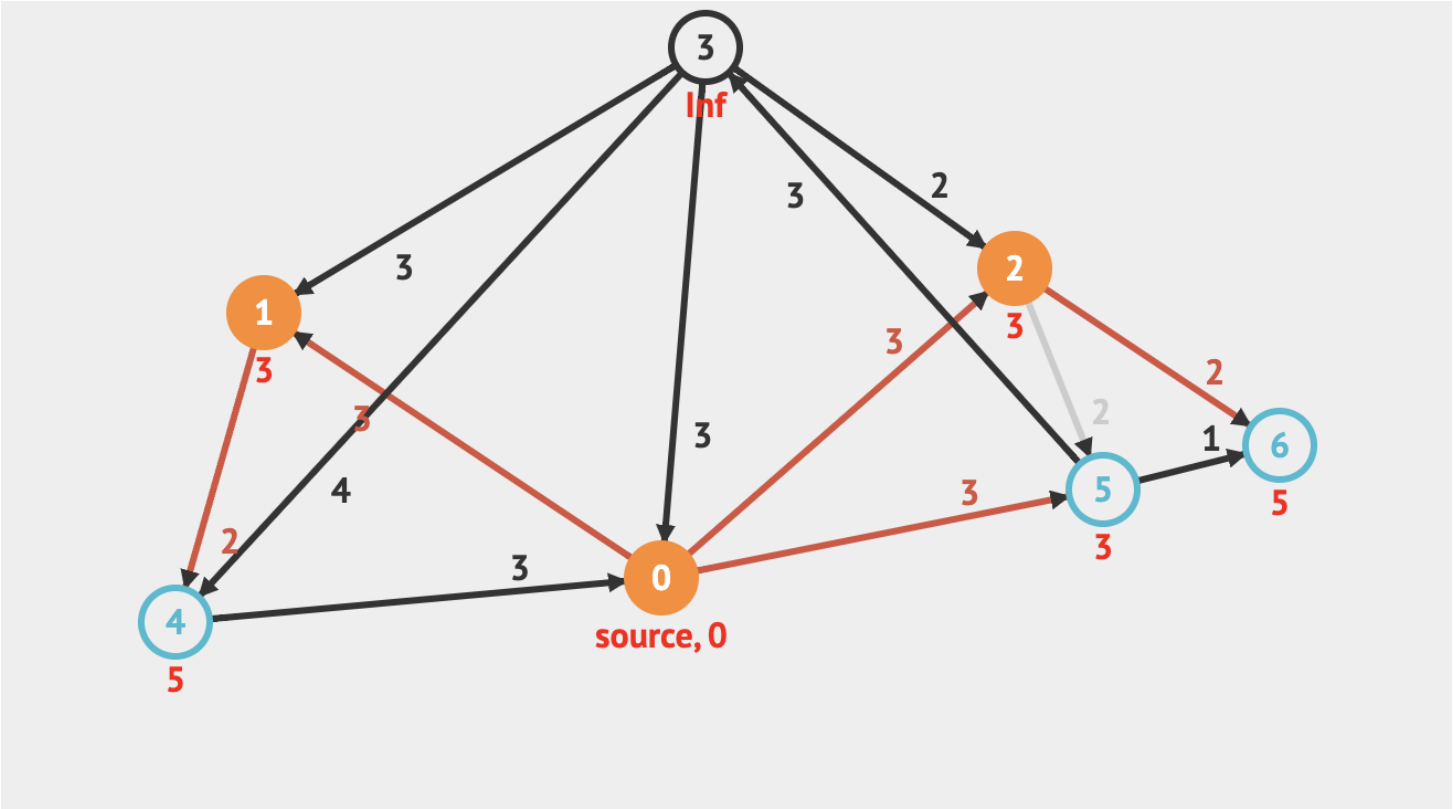
Dijkstra's Algorithm Example



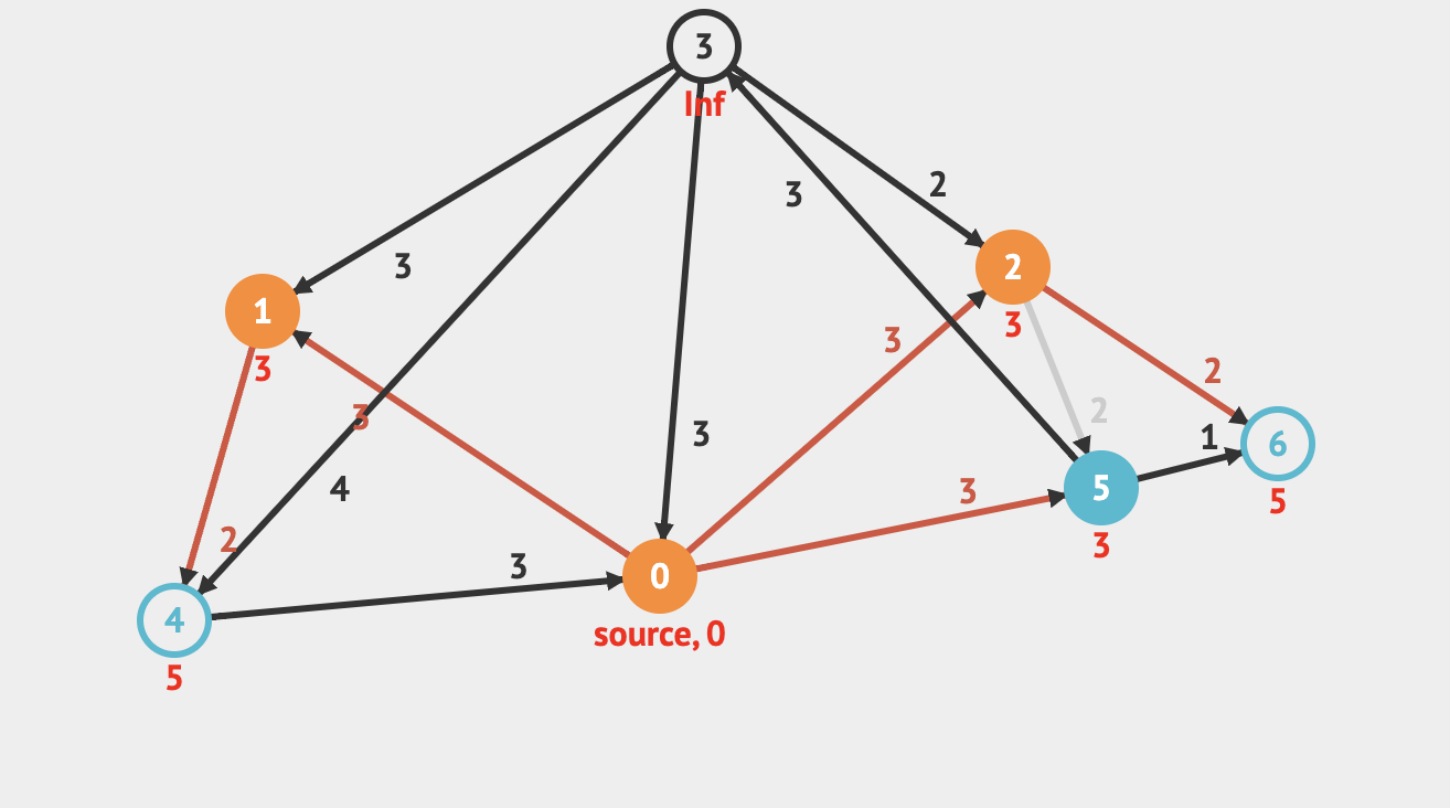
Dijkstra's Algorithm Example



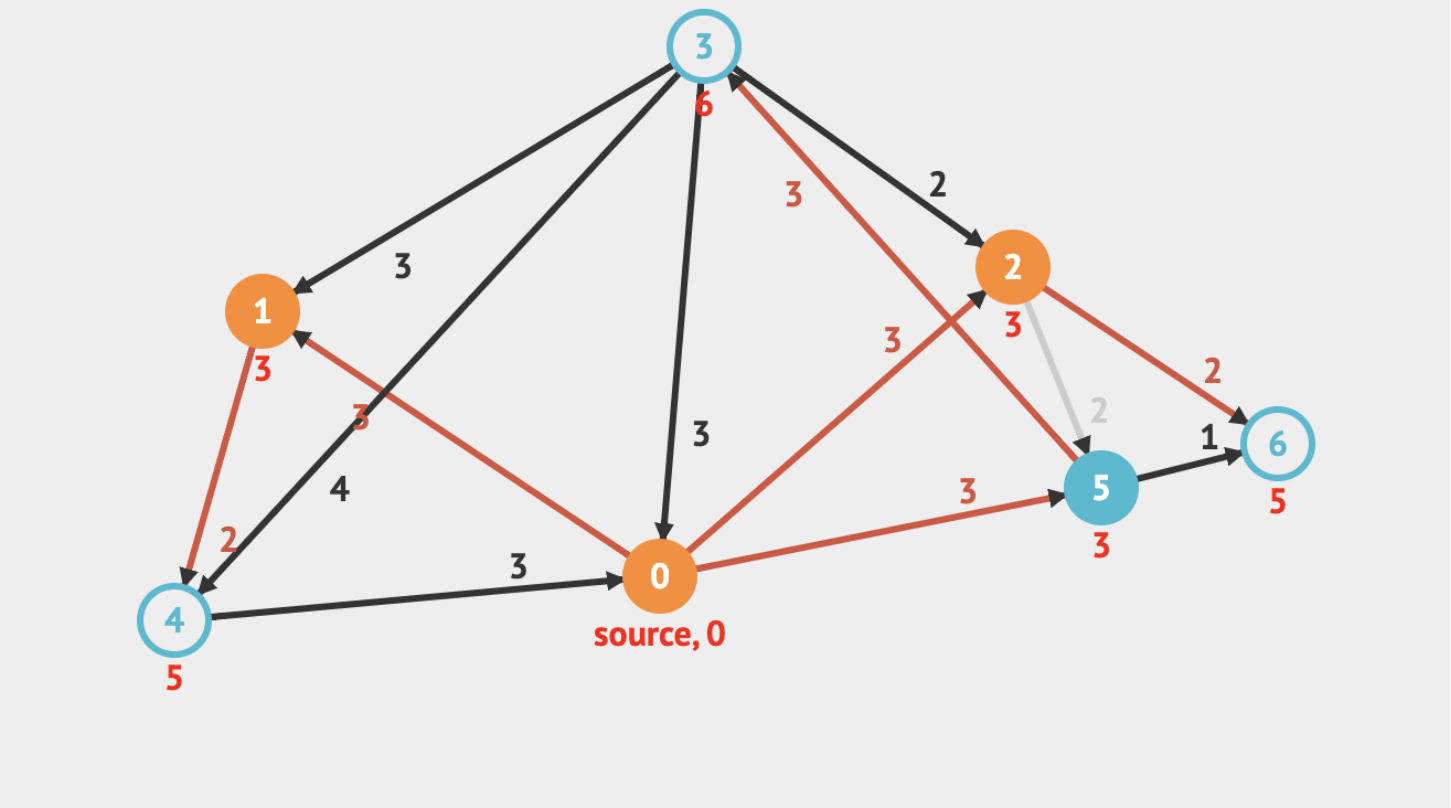
Dijkstra's Algorithm Example



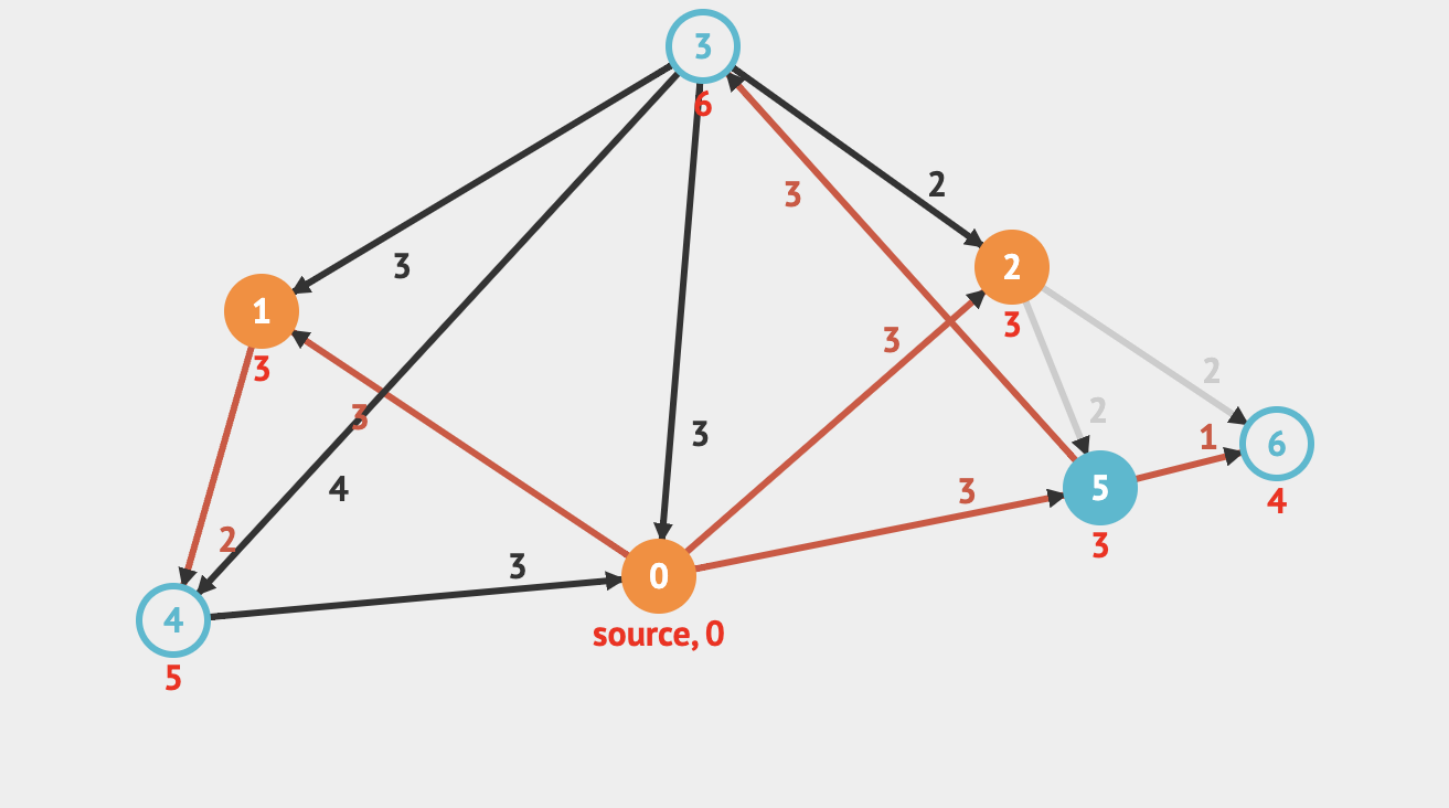
Dijkstra's Algorithm Example



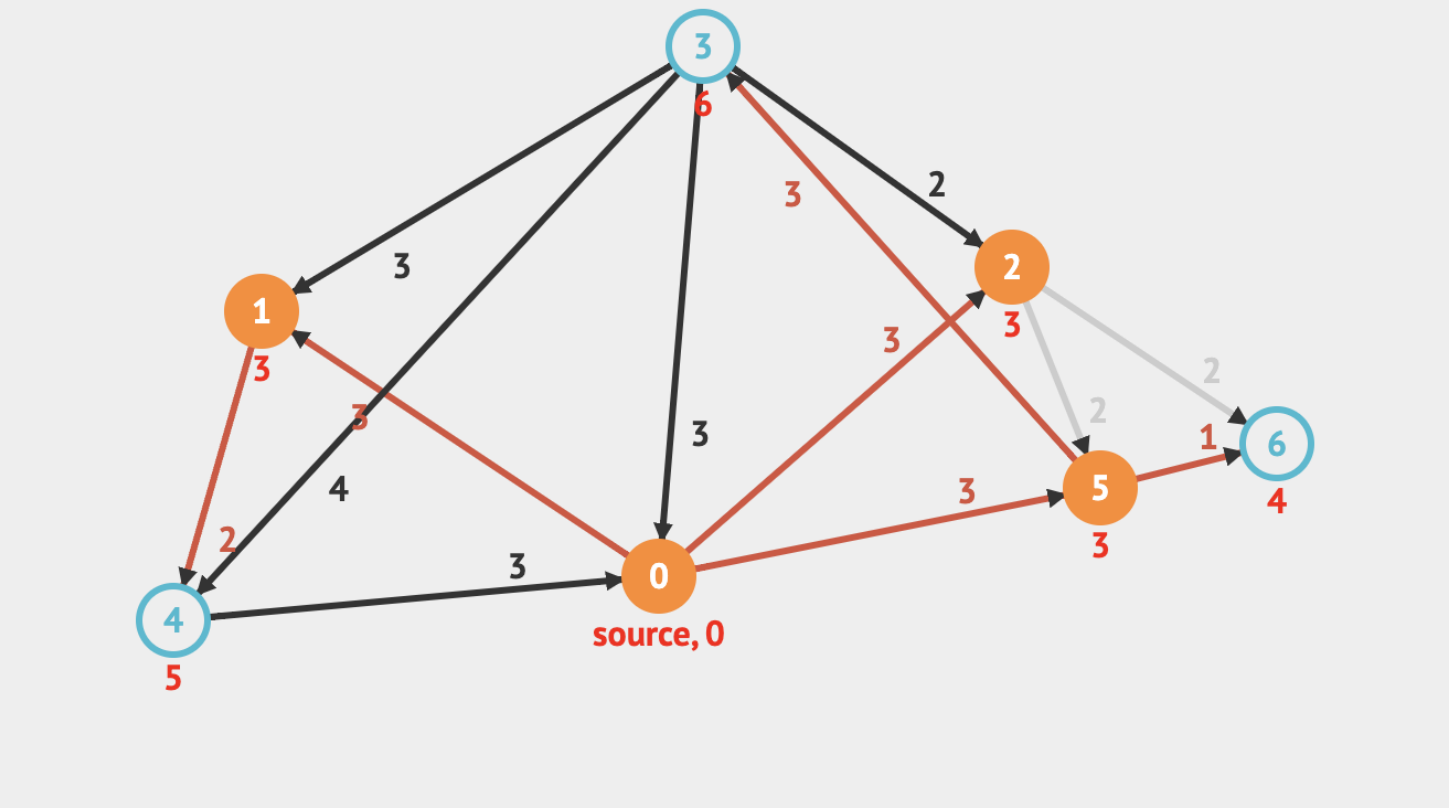
Dijkstra's Algorithm Example



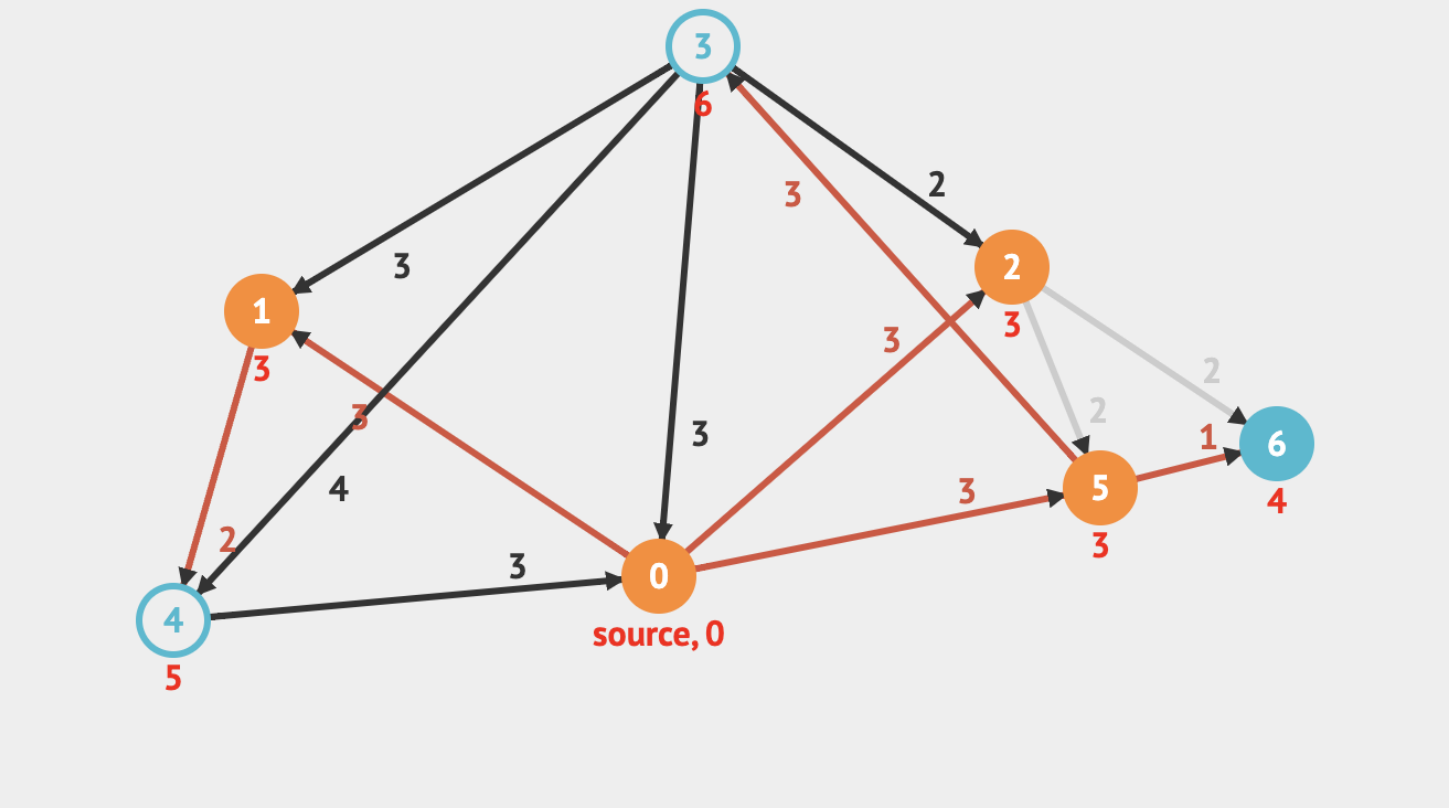
Dijkstra's Algorithm Example



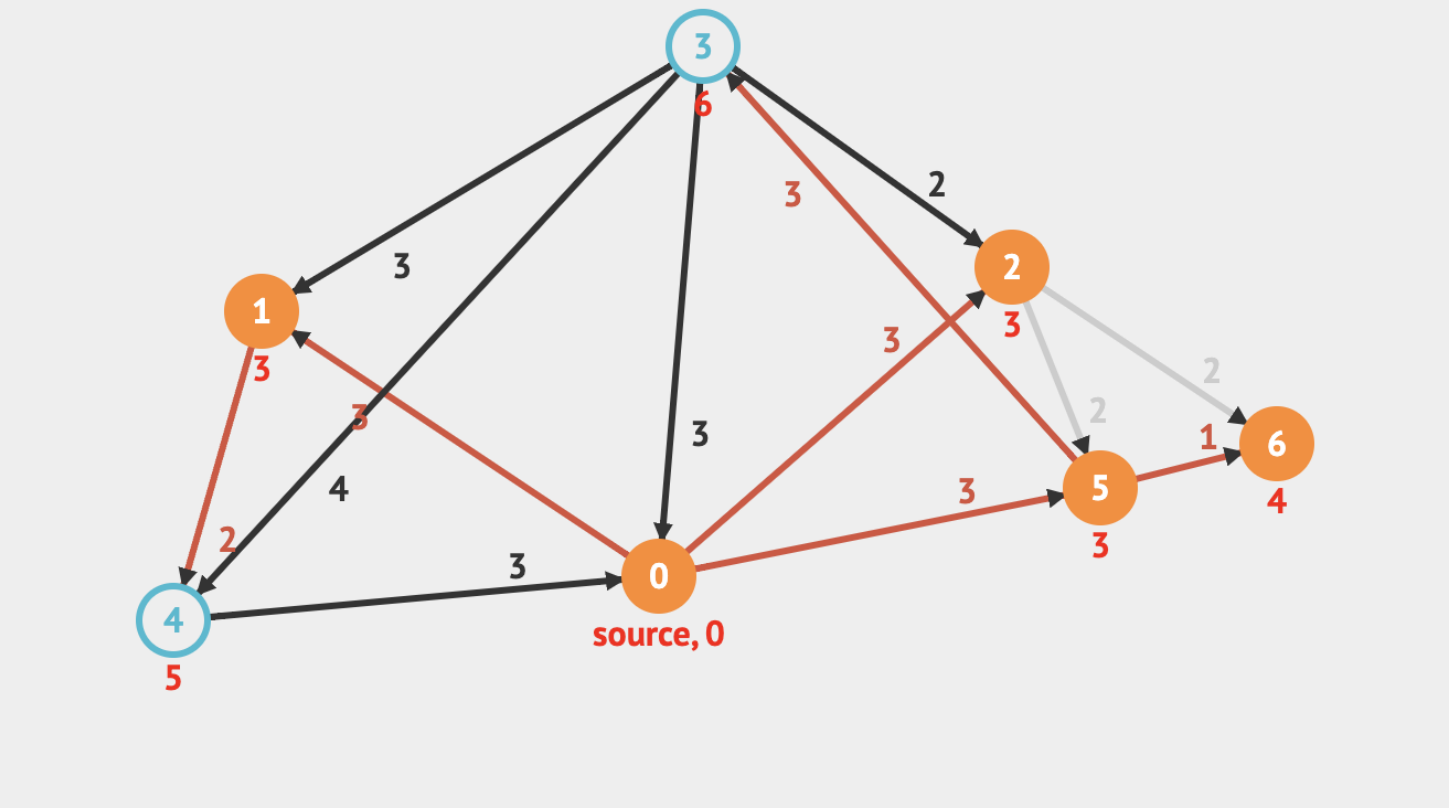
Dijkstra's Algorithm Example



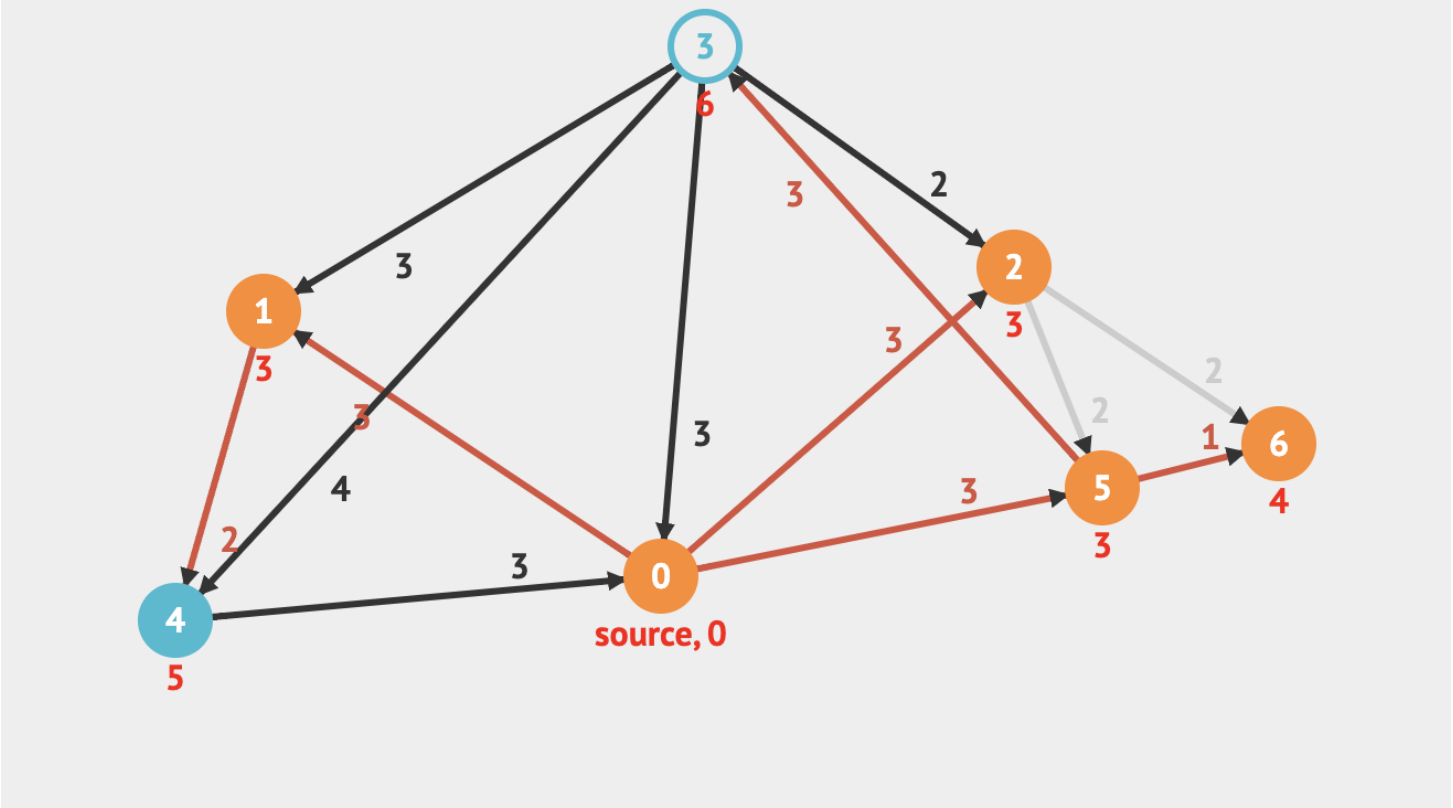
Dijkstra's Algorithm Example



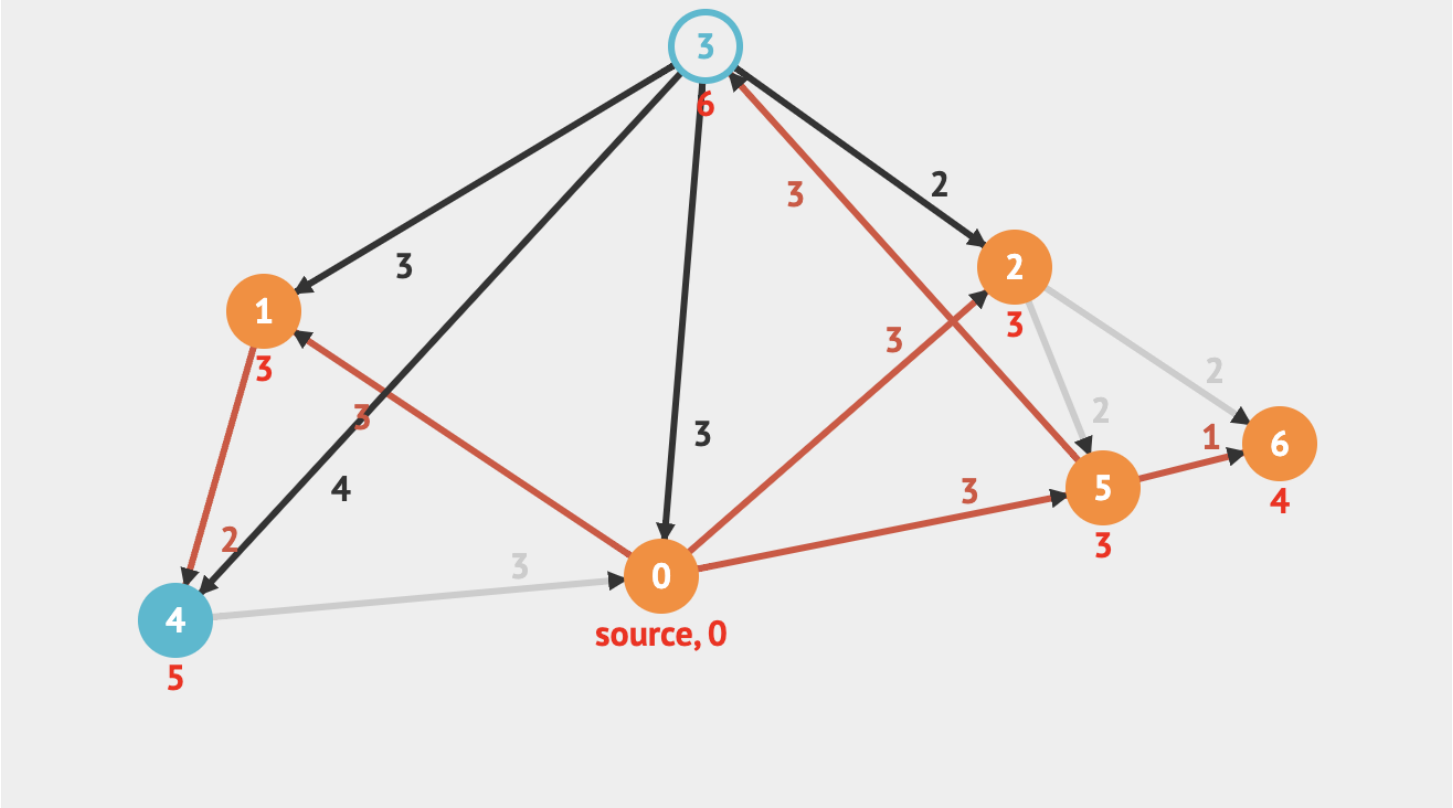
Dijkstra's Algorithm Example



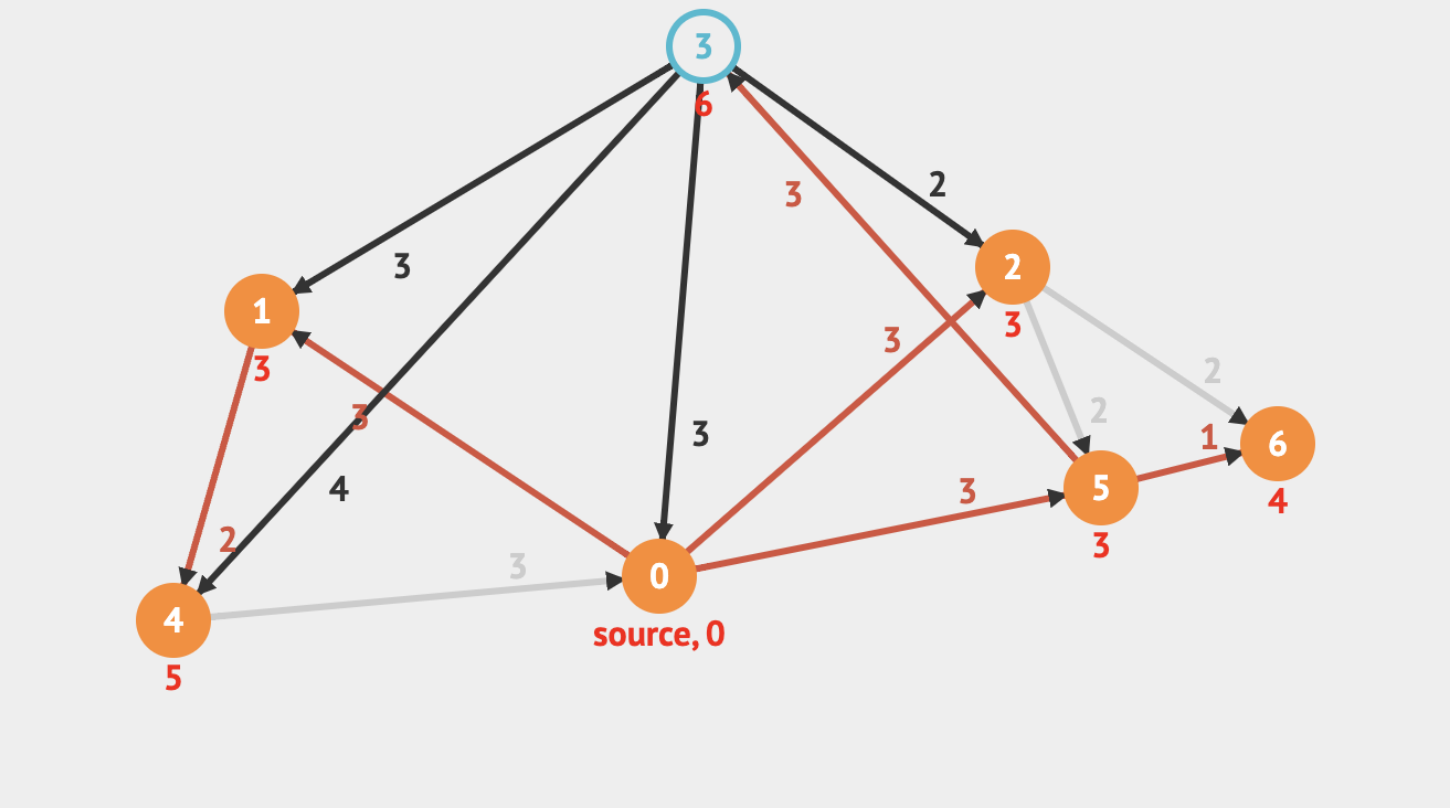
Dijkstra's Algorithm Example



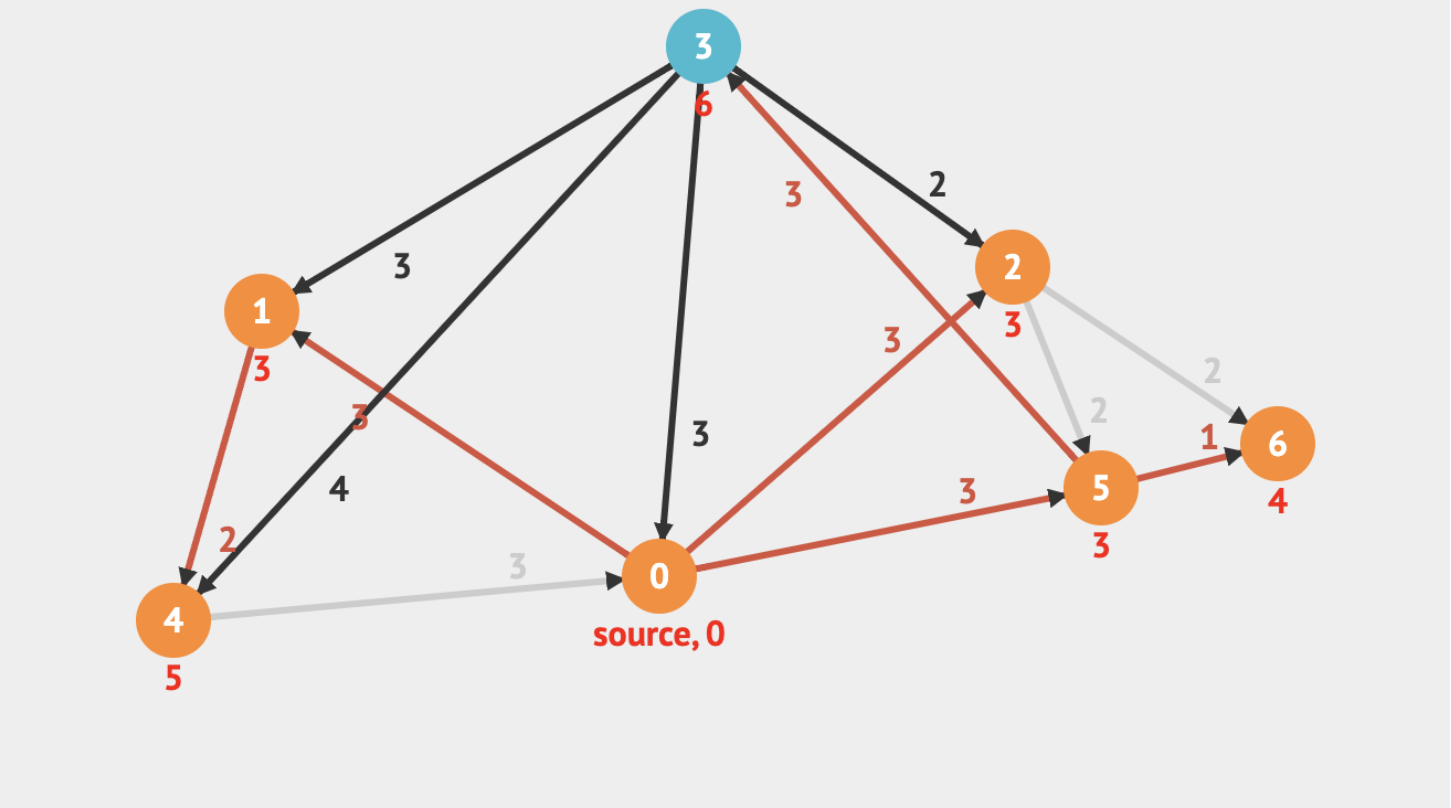
Dijkstra's Algorithm Example



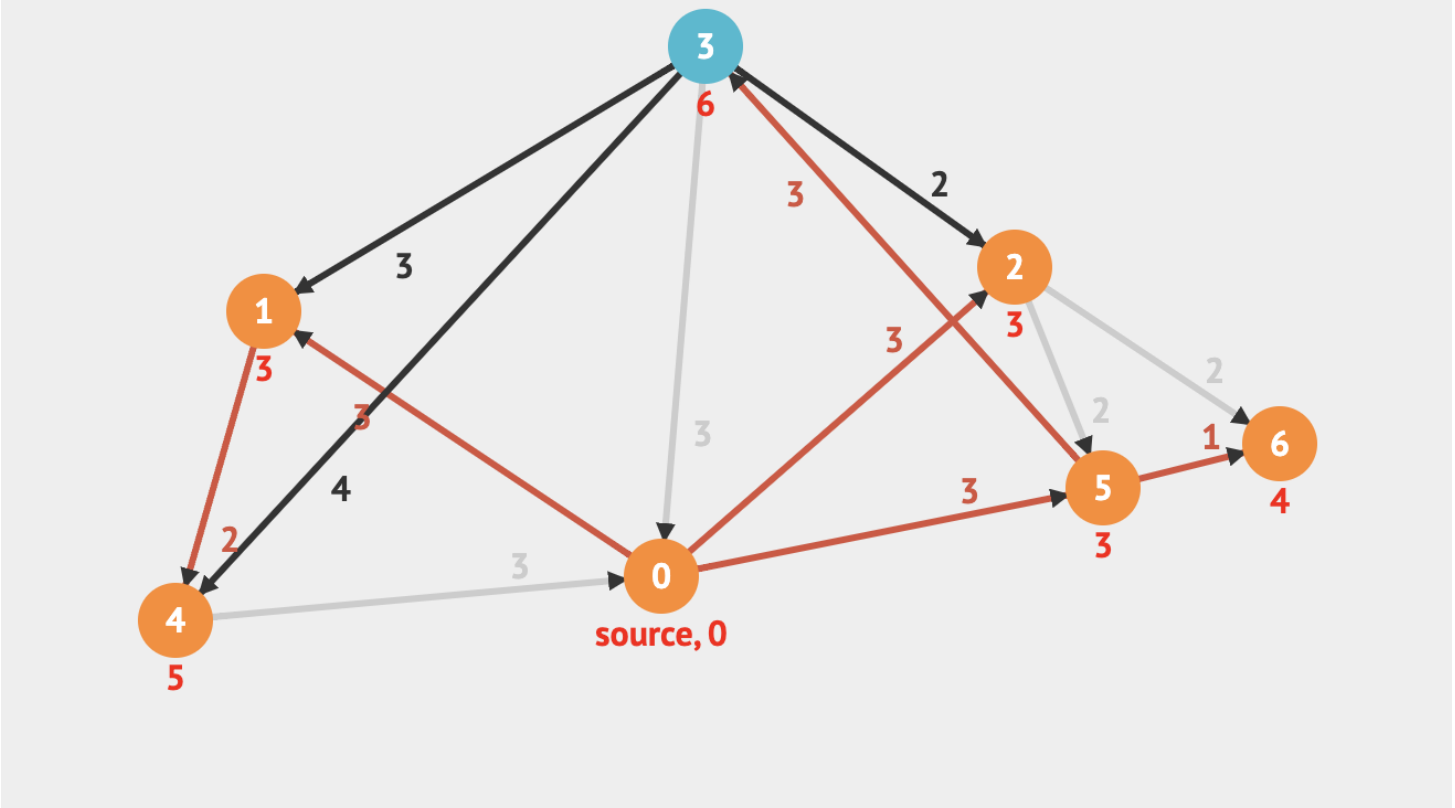
Dijkstra's Algorithm Example



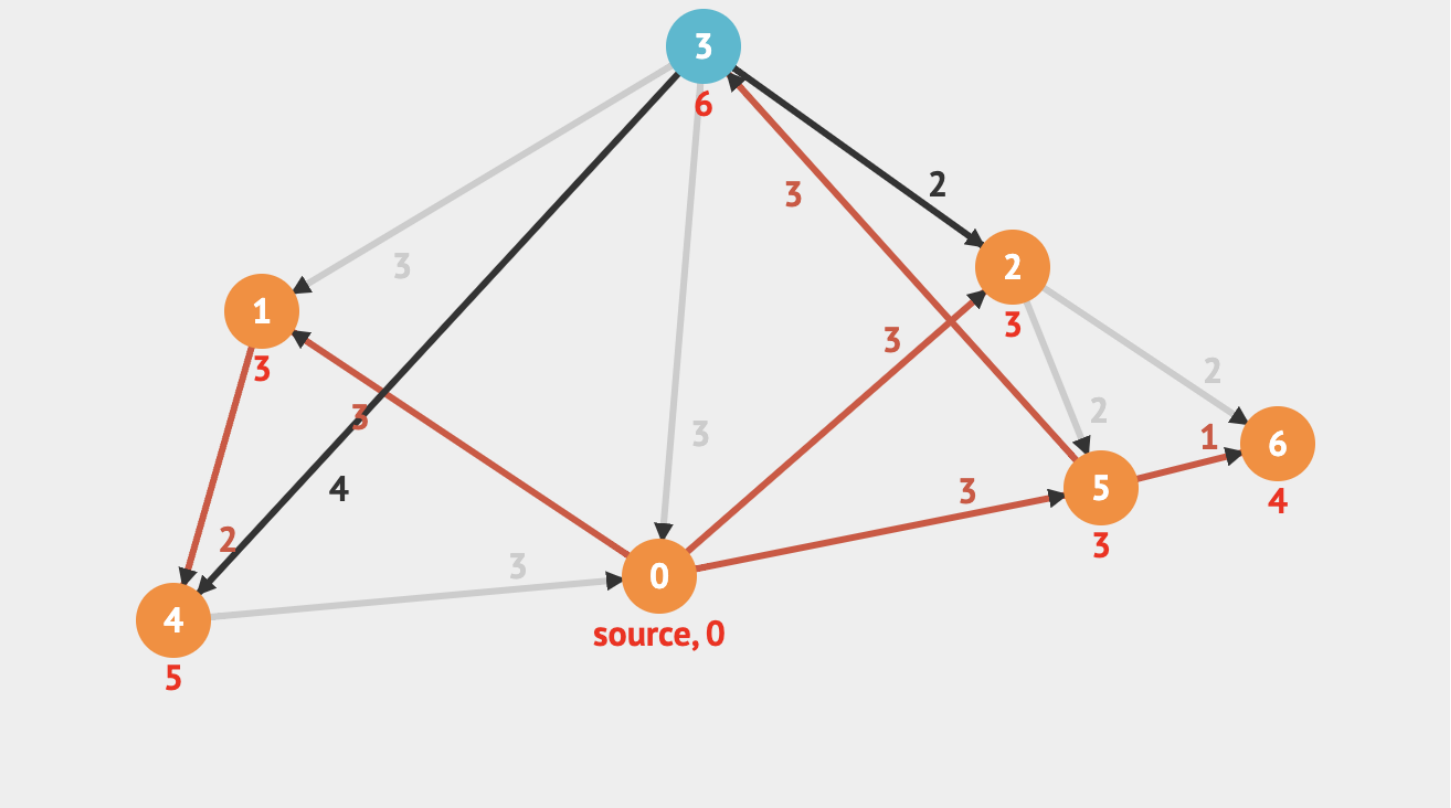
Dijkstra's Algorithm Example



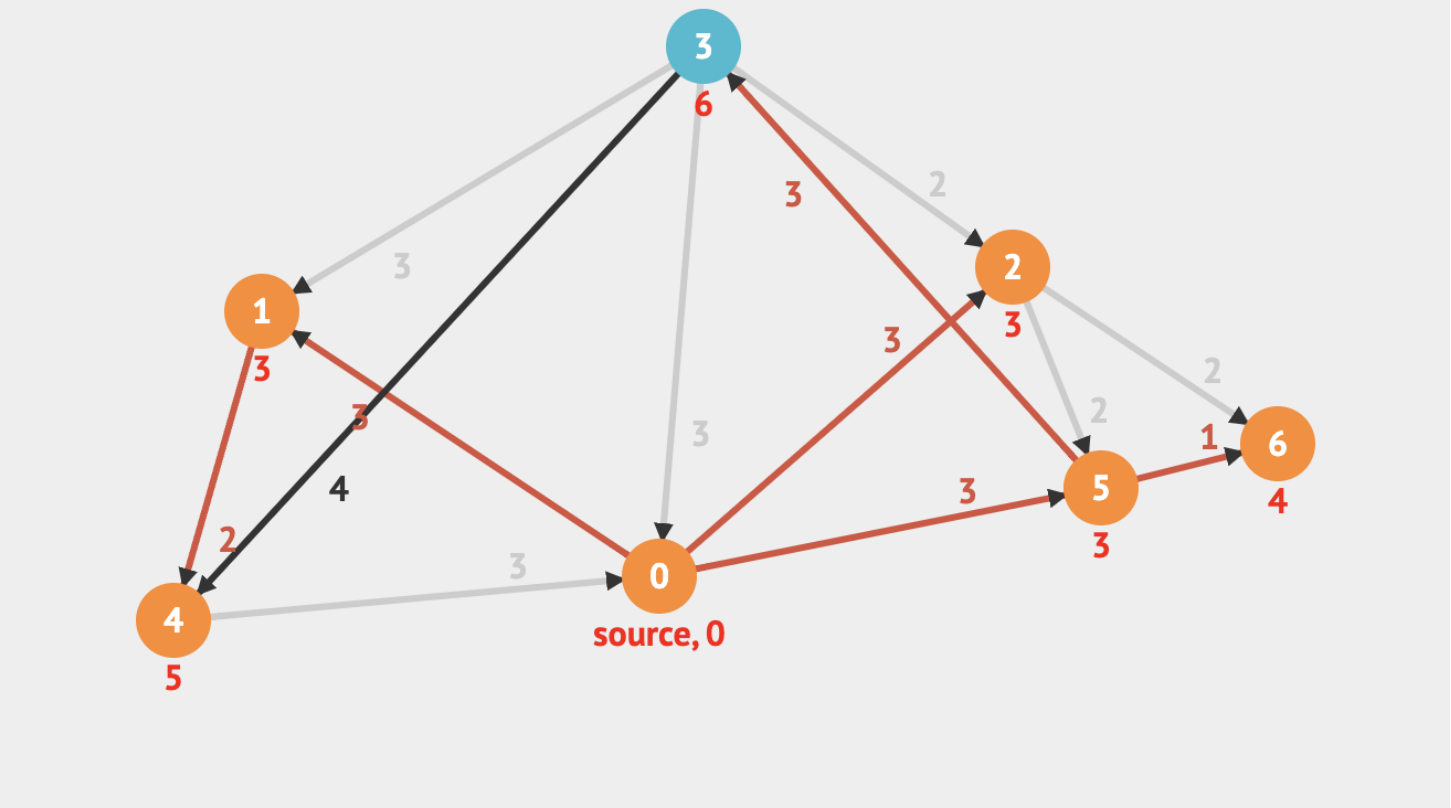
Dijkstra's Algorithm Example



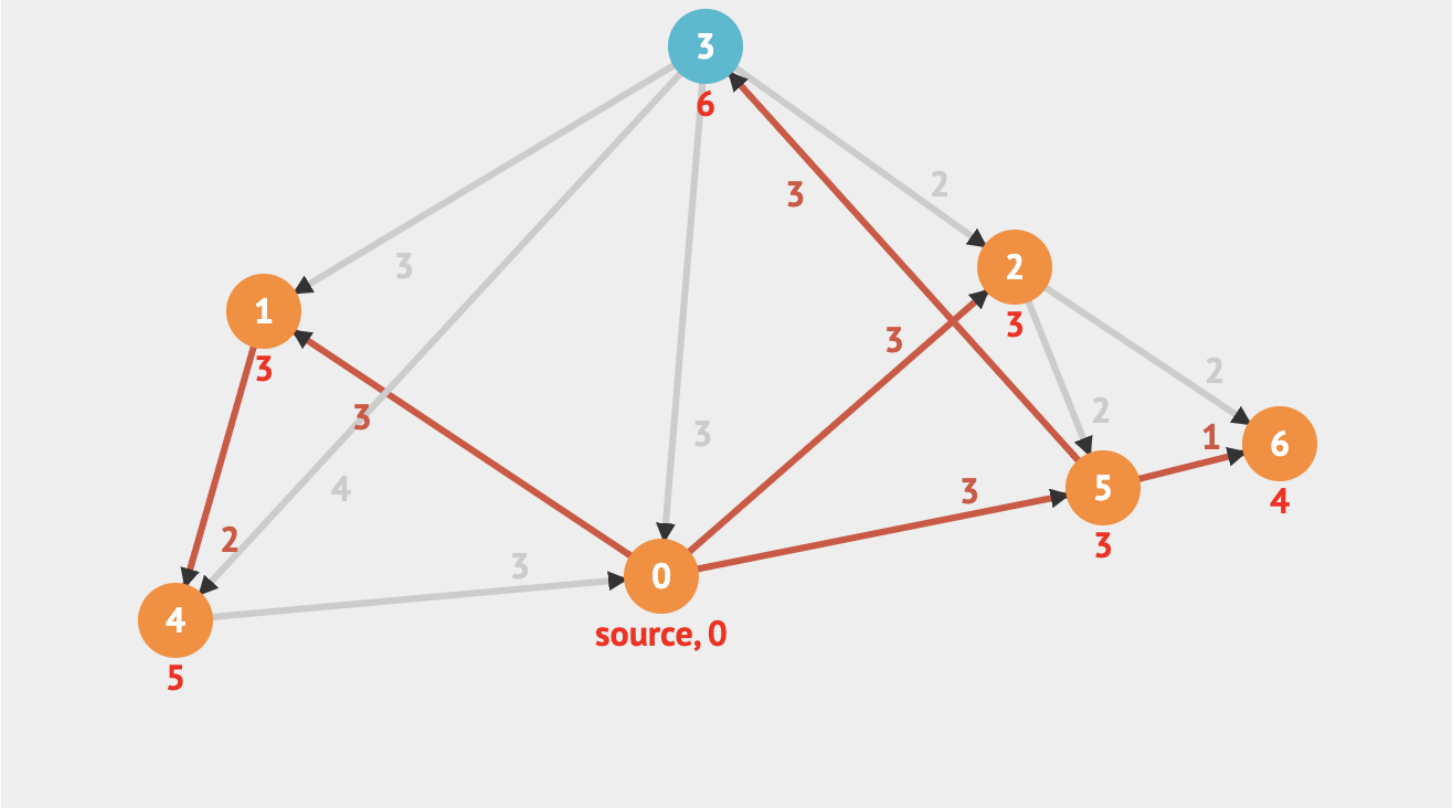
Dijkstra's Algorithm Example



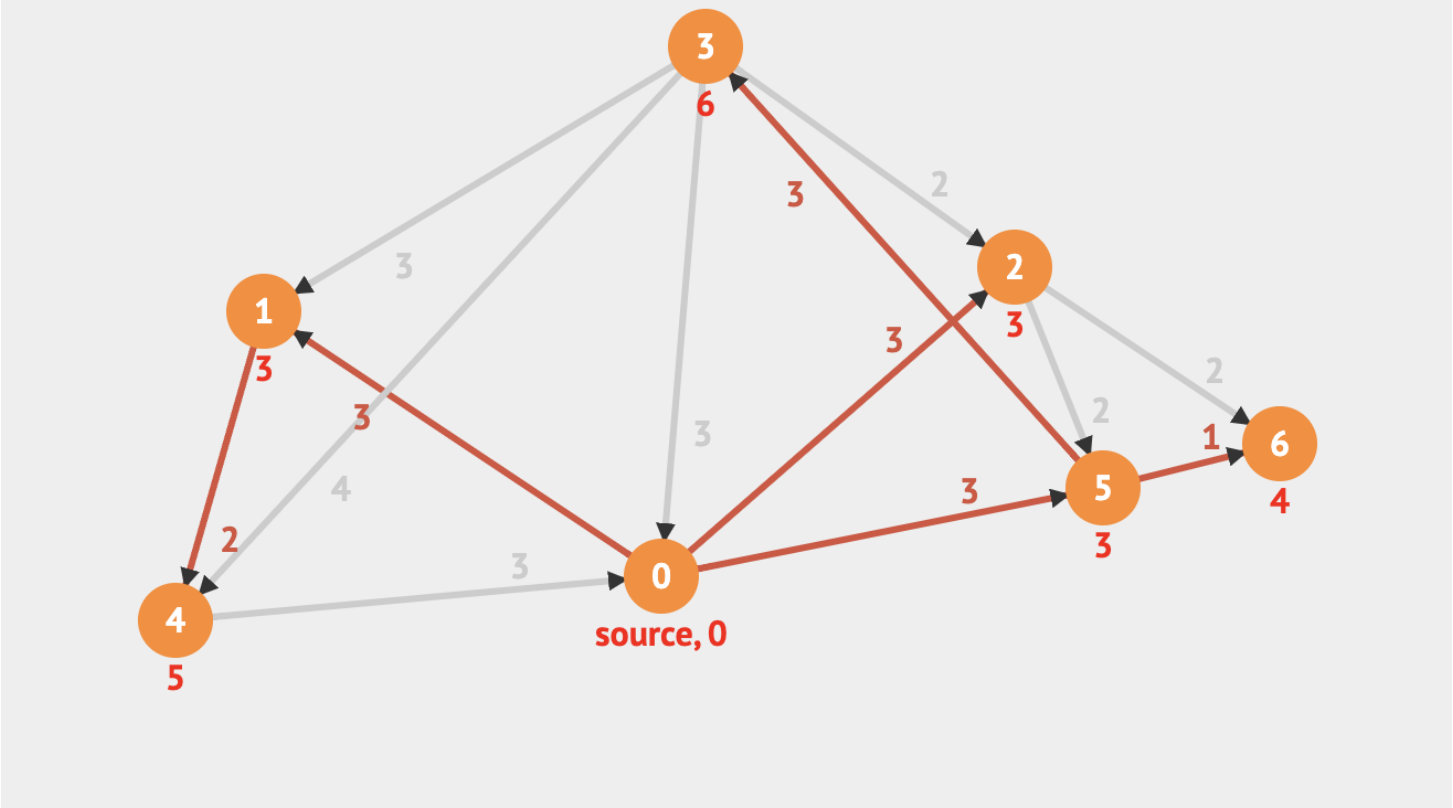
Dijkstra's Algorithm Example



Dijkstra's Algorithm Example



Dijkstra's Algorithm Example



Dijkstra's Algorithm Overview

Goal:

- Find the shortest paths from a single source vertex s to all other vertices $v \in V$.
- Assumes non-negative edge weights ($w(u, v) \geq 0$).

Strategy: Greedy

- Maintain a set S of vertices whose final shortest path distance from s is known.
- Repeatedly select the vertex $u \notin S$ with the minimum shortest path estimate $d[u]$.
- Add u to S and update distance estimates (*relax*) for neighbors of u.

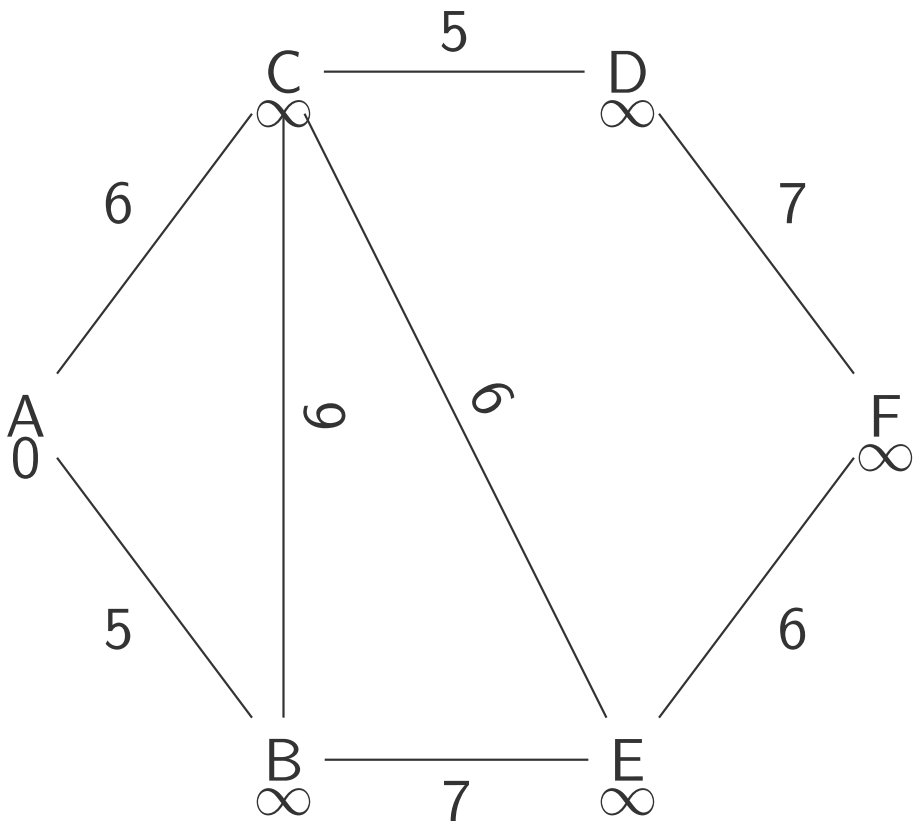
Core Data Structures:

- $d[v]$: Current shortest distance estimate from s to v. ($d[s] = 0$, others ∞ initially).
- $pred[v]$: Predecessor of v on the current shortest path from s.
- Priority Queue (PQ): Stores vertices not in S, keyed by $d[v]$.

Relaxation of edge (u, v): If $d[u] + w(u, v) < d[v]$, then set $d[v] \leftarrow d[u] + w(u, v)$ and $pred[v] \leftarrow u$. Update v in PQ.

Example

Finding shortest paths from source **A**. Distances $d[v]$ shown below nodes. ∞ initially for $v \neq A$.



Initial PQ: $\{$**(A, 0)**$, (B, \infty), (C, \infty), (D, \infty), (E, \infty), (F, \infty)\}$

Example: Step 1 - Extract A

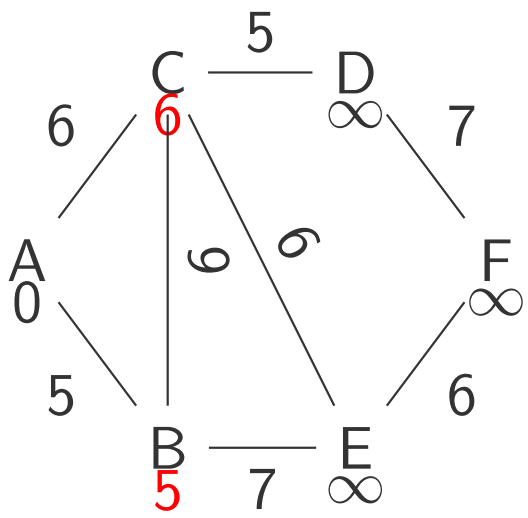
Action: `Extract-Min` $\rightarrow$ **A** $(d = 0)$. $S = \{A\}$. **Relax edges from A:**

- (A,B): $0 + 5 < \infty \implies d[B] = 5, pred[B] = A$.

Priority Queue:

$PQ = \{$**(B, 5)**$, (C, 6), ...\}$

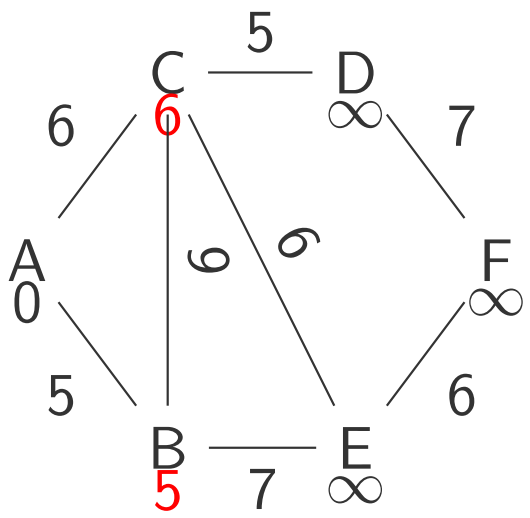
(Only finite entries + ∞ shown)



Example: Step 1 - Extract A

Action: `Extract-Min` $\rightarrow$ **A** $(d = 0)$. $S = \{A\}$. **Relax edges from A:**

- (A,B): $0 + 5 < \infty \implies d[B] = 5, pred[B] = A$.
- (A,C): $0 + 6 < \infty \implies d[C] = 6, pred[C] = A$.



Priority Queue:

$PQ = \{$**(B, 5)**$, (C, 6), ...\}$

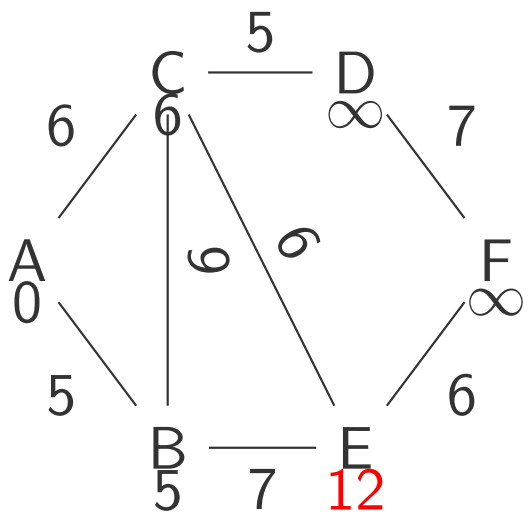
(Only finite entries + ∞ shown)

Example: Step 2 - Extract B

Action: `Extract-Min` $\rightarrow$ **B** ($d = 5$). $S = \{A, B\}$. **Relax edges from B:**

- (B,C): $5 + 6 = 11 \not< d[C](6)$.
  No change.

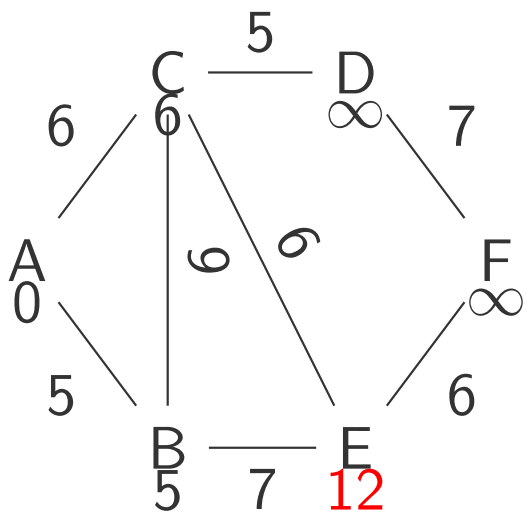
Priority Queue:
$PQ = \{$**(C, 6)**$, (E, 12), ...\}$



Example: Step 2 - Extract B

Action: `Extract-Min` $\rightarrow$ **B** $(d = 5)$. $S = \{A, B\}$. **Relax edges from B:**

- (B,C): $5 + 6 = 11 \not< d[C](6)$. No change.
- (B,E): $5 + 7 = 12 < \infty \implies d[E] = 12, pred[E] = B$.



Priority Queue:

$PQ = \{$**(C, 6)**$, (E, 12), ...\}$

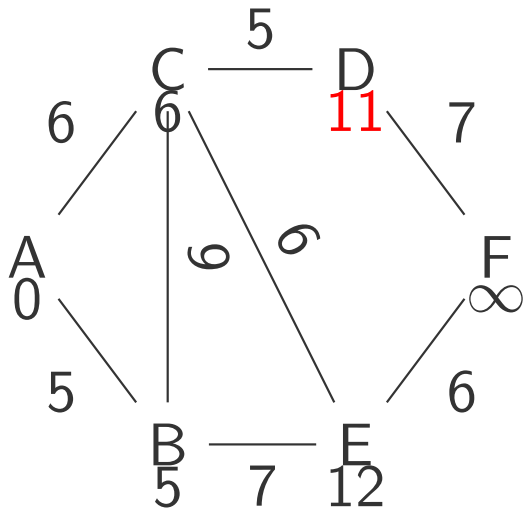
Example: Step 3 - Extract C

Action: `Extract-Min` $\rightarrow$ **C** $(d = 6)$. $S = \{A, B, C\}$. **Relax edges from C:**

- (C,D): $6 + 5 = 11 < \infty \implies d[D] = 11, pred[D] = C$.

Priority Queue:

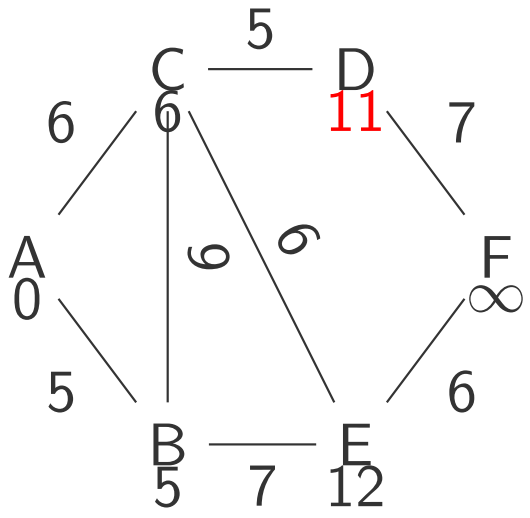
$PQ = \{$**(D, 11)**$, (E, 12), ...\}$



Example: Step 3 - Extract C

Action: `Extract-Min` $\rightarrow$ **C** ($d = 6$). $S = \{A, B, C\}$. **Relax edges from C:**

- (C,D): $6 + 5 = 11 < \infty \implies d[D] = 11, pred[D] = C$.
- (C,E): $6 + 6 = 12 \not< d[E](12)$. No change.



Priority Queue:

$PQ = \{$**(D, 11)**$, (E, 12), ...\}$

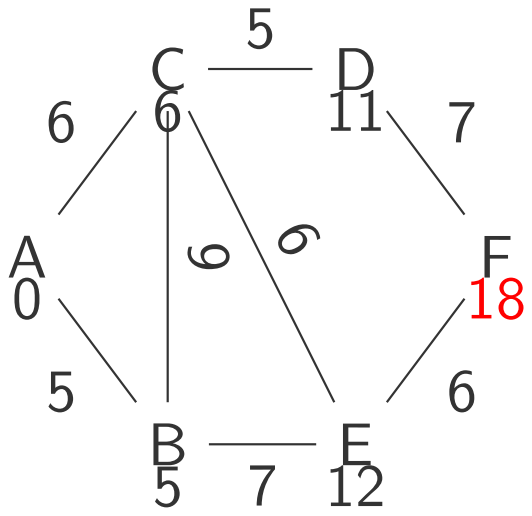
Example: Step 4 - Extract D

Action: `Extract-Min` $\rightarrow$ **D** ($d = 11$). $S = \{A, B, C, D\}$. **Relax edges from D:**

- (D,F): $11 + 7 = 18 < \infty \implies d[F] = 18, pred[F] = D$.

Priority Queue:

$PQ = \{$**(E, 12)**$, (F, 18)\}$



Example: Step 5 - Extract E

Action: `Extract-Min` $\rightarrow$ **E**

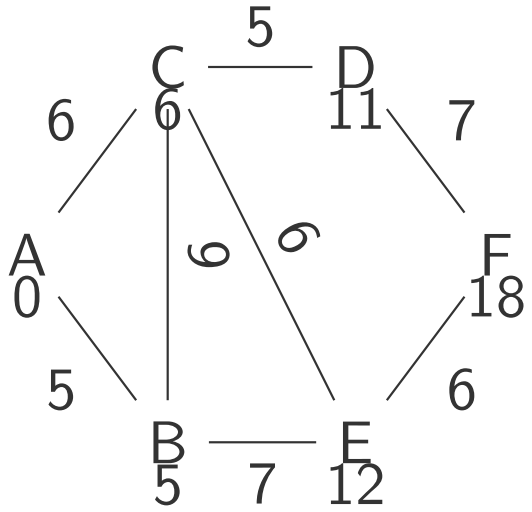
$(d = 12)$. $S = \{A, B, C, D, E\}$.

Relax edges from E:

- (E,F): $12 + 6 = 18 \not< d[F](18)$.
  No change.

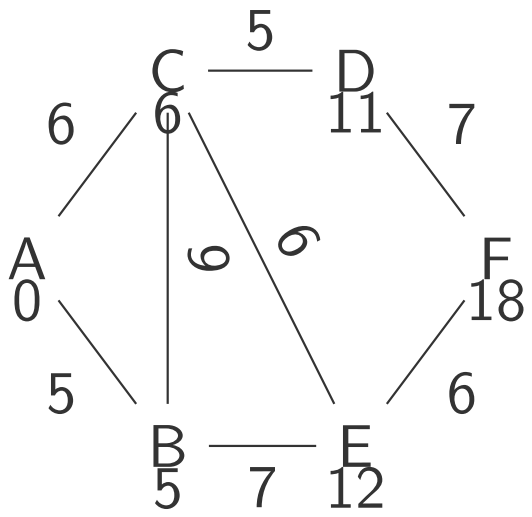
Priority Queue:

$PQ = \{$**(F, 18)**$\}$



Example: Step 6 - Extract F

Action: `Extract-Min` $\rightarrow$ **F** ($d = 18$). $S = \{A..F\}$. **Relax Edges:** None (no unvisited neighbors).



Priority Queue: $PQ = \{\}$

(Empty) **Algorithm**

Terminates.

Dijkstra's Algorithm: Runtime Comparison

Runtime depends on efficiently finding the unvisited node with minimum distance ($d[u]$) and updating neighbor distances (`Decrease-Key` in PQ). Let V = Vertices, E = Edges.

Implementation	**Data Structure**	**Find Min**	**Decrease-Key**
Naive	Array Scan	$O(V)$	$O(1)$
Standard	Binary Heap PQ	$O(\log V)$	$O(\log V)$
Advanced	Fibonacci Heap PQ	$O(\log V)^*$	$O(1)^*$

*Amortized cost

Dijkstra's Algorithm: Proof of Correctness

- **Inductive Proof Outline**:
  - We prove by induction that at each iteration of the algorithm, the set C of all visited (marked) vertices has the following properties:
    1. For any $u \in C$ and $v \in V - C$, $\text{dist}(s, u) \leq \text{dist}(s, v)$.
    2. For every $v \in C$, $d[v] = \text{dist}(s, v)$, i.e., $d[v]$ represents the correct shortest distance from s.
- **Base Case (Iteration 0)**:
  - Initially, only s is marked in C.
  - $d[s] = \text{dist}(s, s) = 0$, and s is the closest vertex to itself.
  - Properties hold trivially for the initial state.
- **Inductive Step**:
  - Assume the properties hold after i iterations.
  - In the $(i + 1)$-th iteration, let $e = (u, v)$ be the edge removed from the priority queue with minimum $d[v]$.
  - If v is already marked, ignore e (set C remains unchanged).
  - If v is unmarked:
    1. $d[v] = d[u] + w(u, v) = \text{dist}(s, u) + w(u, v) = \text{dist}(s, v)$.
    2. v is marked and added to C, maintaining $d[v] = \text{dist}(s, v)$.

Proof of Correctness (continued)

- **Maintaining Closest Distance**:
  - For every unvisited vertex $w \in V - C$, $\text{dist}(s, w) \geq \text{dist}(s, v)$.
  - e has the minimum value in the priority queue, ensuring v is the closest unvisited vertex to s.
  - No shorter path exists to v through unvisited vertices.
- **Conclusion**:
  - By induction, $d[v] = \text{dist}(s, v)$ for all $v \in V$ by the end of the algorithm.
  - Dijkstra's algorithm thus computes the correct shortest path distances from s.

Distance to Hub

Problem:

- **Input**: Graph $G = (V, E)$, subset $H \subseteq V$ of hubs.
- **Output**: For each $v \in V$, find the shortest distance from a hub $h \in H$.

Distance to Hub

Problem:

- **Input**: Graph $G = (V, E)$, subset $H \subseteq V$ of hubs.
- **Output**: For each $v \in V$, find the shortest distance from a hub $h \in H$.

Naive Solution:

- Run Dijkstra from each $h \in H$.
- Compute $\text{dist}(v) = \min_{h \in H} \text{dist}(h, v)$.
- **Runtime**: $O(|H| \cdot (|E| + |V| \log |V|))$.

Better Solution:

- Add a dummy source s with edges (s, h) of weight 0 for $h \in H$.
- Run Dijkstra from s.
- **Runtime**: $O(|E| + |V| \log |V|)$.

Swim in Rising Water

Problem Statement:

- You are given an $N \times N$ grid where each cell (i, j) has an elevation height grid$[i][j]$.
- You start at the top-left cell $(0, 0)$ and must reach the bottom-right cell $(N-1, N-1)$.
- At time t, you can move between adjacent cells (i, j) and (k, l) if both grid$[i][j] \leq t$ and grid$[k][l] \leq t$.
- Minimize the time t required to reach the destination.

Approach:

- Model the grid as a graph where each cell is a node.
- Use Dijkstra's algorithm:
  - Treat the cost of moving between two cells (i, j) and (k, l) as the maximum elevation of the two cells:

$$\text{cost} = \max(\text{grid}[i][j], \text{grid}[k][l])$$

  - Minimize the maximum elevation encountered along the path.

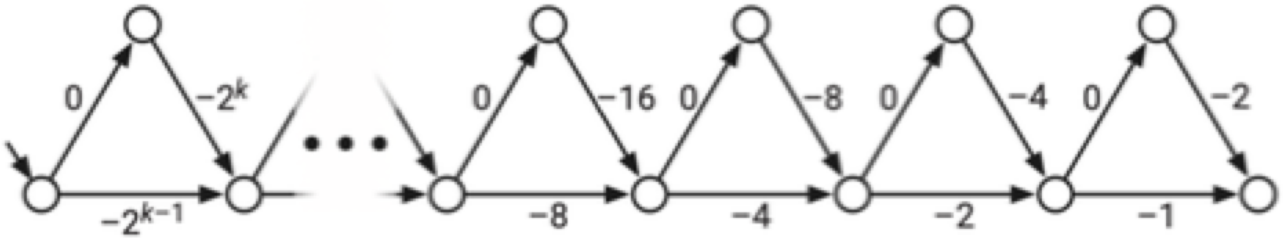
A Different Version of Dijkstra

- Similar to Breadth-First Search (BFS), but replaces the FIFO queue with a priority queue.
- Steps:
  - Initialize $\text{dist}(s) = 0$, $\text{dist}(v) = \infty$ for all $v \neq s$.
  - Use a priority queue to manage vertices by their current shortest distances.
  - Extract u with the smallest distance and relax all outgoing edges $u \to v$.
  - Update or insert v into the queue with its new distance.

Negative edges

Example of Exponential Behavior:

- Certain graph families (Shier and Witzgall examples) force exponential relaxations:
  - $\Theta(2^{V/2})$ relaxations for simpler cases.
  - $\Theta(2^{V})$ relaxations in the worst-case scenario.
- This happens due to repeated re-insertion of vertices into the priority queue.



Bellman-Ford Algorithm: Overview

- **Purpose**: Computes shortest paths from source s to all other vertices in G.
- Handles graphs with negative edge weights, as long as no negative weight cycles exist.
- **Initialization**:
  - Set $d[s] = 0$ (distance from source to itself is zero).
  - For all $v \neq s$, set $d[v] = +\infty$ (initially assume all other vertices are unreachable).

Bellman-Ford Algorithm: Steps

1. For $n-1$ iterations:
   - For each edge (u, v) with weight $w(u, v)$:
   - If $d[v] > d[u] + w(u, v)$:
     - Update $d[v] = d[u] + w(u, v)$.
   - This "relaxes" edge (u, v) by updating $d[v]$ if a shorter path is found.
2. After $n-1$ iterations, each vertex's distance represents the shortest path from s.

The Bellman-Ford Algorithm: High Level

We maintain:

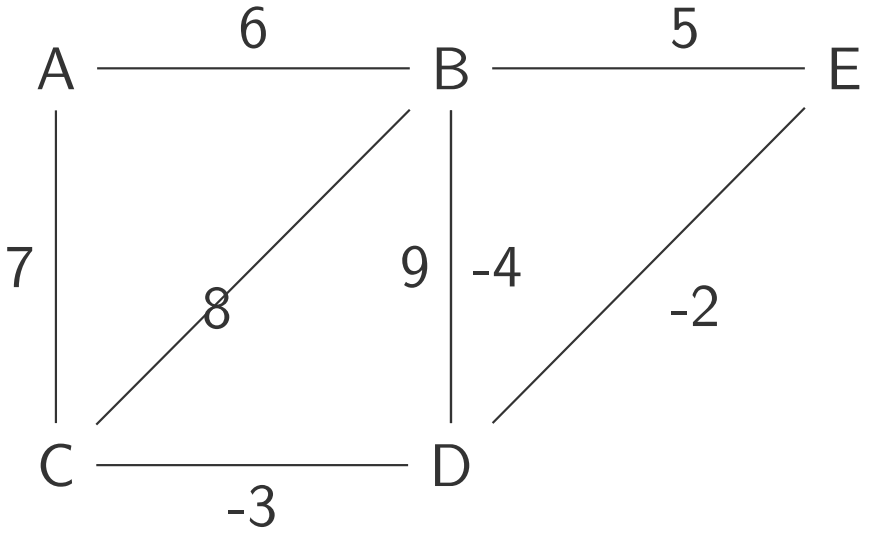
- $d[v]$: Current shortest distance estimate from s to v.
- $\pi[v]$: Predecessor of v in the shortest path found so far.

Steps:

1. **Initialization:**
   - Set $d[s] = 0$.
   - Set $d[v] = \infty$ for all $v \neq s$.
   - Set $\pi[v] = \text{NIL}$ for all v.
2. **Relaxation Passes:** Repeat $V - 1$ times:
   - For each edge $(u, v) \in E$:
   - **Relax** the edge: If $d[u] + w(u, v) < d[v]$, then update:
     - $d[v] = d[u] + w(u, v)$
     - $\pi[v] = u$
3. **Negative Cycle Check:** For each edge $(u, v) \in E$:
   - If $d[u] + w(u, v) < d[v]$, then a negative cycle reachable from s exists. Report it and terminate (or handle as needed).

Example 1: Graph Setup (No Negative Cycle)

Goal: Find shortest paths from source **A**.

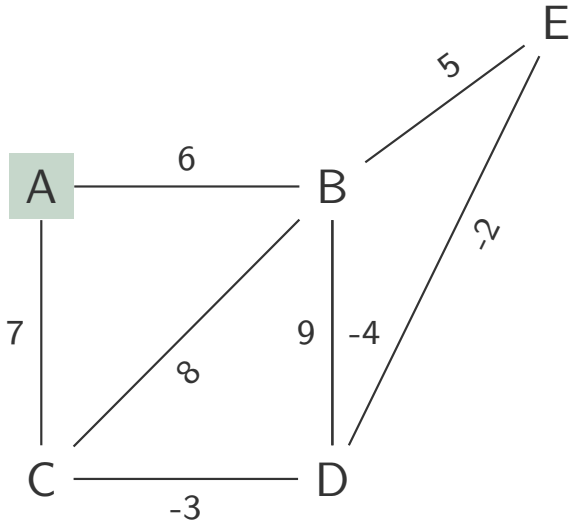


$V = 5$, $E = 8$. We need

$V - 1 = 4$ relaxation passes, plus one check pass.

Example 1: Initialization (Pass 0)

Goal: Set initial distances. $d[A] = 0$, others ∞. Predecessors $\pi[v]$ $=$ NIL.

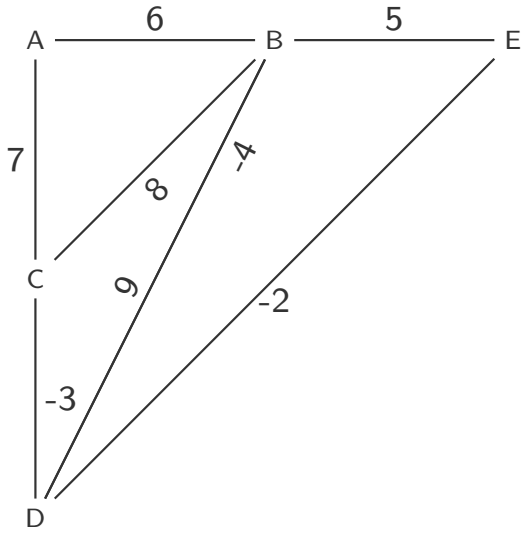


Distances (d) / Predecessors (π)

Vertex	$d[v]$	$\pi[v]$
A	0	NIL
B	∞	NIL
C	∞	NIL
D	∞	NIL
E	∞	NIL

Example 1: After Pass 1

Action: Relax all 8 edges. Edges from A update B and C.



Distances / Predecessors

Vertex	$d[v]$	$\pi[v]$
A	0	NIL
B	6	A
C	7	A
D	∞	NIL
E	∞	NIL

Updates:

Relax (A,B):

$d[A] + w(A, B) = 0 + 6 = 6 < \infty.$

Update $d[B] = 6, \pi[B] = A.$

Relax (A,C):

$d[A] + w(A, C) = 0 + 7 = 7 < \infty.$

Update $d[C] = 7, \pi[C] = A.$

(Other edges relaxed, no updates as source distance is ∞).

Example 1: Setup

Graph Details

- Vertices: A, B, C, D, E $\quad (|V| = 5)$
- Source: A
- Edges (Weight):
  - (A,B): 6, (A,C): 7
  - (B,C): 8, (B,D): -4, (B,E): 5
  - (C,D): -3
  - (D,B): 9
  - (E,D): -2
- $|E| = 8$

Bellman-Ford Parameters

- Passes needed: $|V| - 1 = 4$
- Check pass: 1 (Total 5 passes)
- Relaxation Order (Example): (A,B), (A,C), (B,C), (B,D), (B,E), (C,D), (D,B), (E,D)

Example 1: Initialization Pass 1

Initialization (Pass 0)

- $d = \{A : 0, B : \infty, C : \infty, D : \infty, E : \infty\}$
- $\pi = \{A : \text{NIL}, B : \text{NIL}, C : \text{NIL}, D : \text{NIL}, E : \text{NIL}\}$

Pass 1 Relaxations

- Relax (A,B): $0 + 6 < \infty \implies d[B] = 6, \pi[B] = A$
- Relax (A,C): $0 + 7 < \infty \implies d[C] = 7, \pi[C] = A$
- Relax (B,C): $6 + 8 \not< 7$
- Relax (B,D): $6 - 4 < \infty \implies d[D] = 2, \pi[D] = B$
- Relax (B,E): $6 + 5 < \infty \implies d[E] = 11, \pi[E] = B$
- Relax (C,D): $7 - 3 = 4 \not< d[D] = 2$
- Relax (D,B): $2 + 9 \not< d[B] = 6$
- Relax (E,D): $11 - 2 = 9 \not< d[D] = 2$

End of Pass 1 State

- $d = \{A : 0, B : 6, C : 7, D : 2, E : 11\}$
- $\pi = \{A : \text{NIL}, B : A, C : A, D : B, E : B\}$

Example 1: Passes 2, 3, 4

Pass 2 Relaxations Checking all edges with Pass 1 values:

- (A,B): $0 + 6 \not< 6$
- (A,C): $0 + 7 \not< 7$
- (B,C): $6 + 8 \not< 7$
- (B,D): $6 - 4 \not< 2$
- (B,E): $6 + 5 \not< 11$
- (C,D): $7 - 3 \not< 2$
- (D,B): $2 + 9 \not< 6$
- (E,D): $11 - 2 \not< 2$

Result: No updates occurred in Pass 2. Pass 3 ($|V| - 2$)

- Since Pass 2 had no updates, Pass 3 will also have **no updates**.

Pass 4 ($|V| - 1$)

- Since Pass 3 had no updates, Pass 4 will also have **no updates**.

End of Pass 4 State (Final Candidate Distances)

- $d = \{A : 0, B : 6, C : 7, D : 2, E : 11\}$
- $\pi = \{A : \text{NIL}, B : A, C : A, D : B, E : B\}$

Example 1: Pass 5 (Negative Cycle Check)

Action: Relax all edges using final candidate distances from Pass 4.

- Check (A,B): $0 + 6 \not< 6$
- Check (A,C): $0 + 7 \not< 7$
- Check (B,C): $6 + 8 \not< 7$
- Check (B,D): $6 - 4 \not< 2$
- Check (B,E): $6 + 5 \not< 11$
- Check (C,D): $7 - 3 = 4 \not< 2$
- Check (D,B): $2 + 9 = 11 \not< 6$
- Check (E,D): $11 - 2 = 9 \not< 2$

Result

- No relaxation improved any distance $d[v]$.
- No negative-weight cycle reachable from source A detected.

Example 1: Conclusion

Final Shortest Paths The Bellman-Ford algorithm terminated successfully without detecting negative cycles reachable from A. The shortest path distances and predecessors found after Pass 4 are:

- $d[A] = 0$, $\pi[A] = \text{NIL}$ (Path: A)
- $d[B] = 6$, $\pi[B] = A$ (Path: A $\rightarrow$ B)
- $d[C] = 7$, $\pi[C] = A$ (Path: A $\rightarrow$ C)
- $d[D] = 2$, $\pi[D] = B$ (Path: A $\rightarrow$ B $\rightarrow$ D)
- $d[E] = 11$, $\pi[E] = B$ (Path: A $\rightarrow$ B $\rightarrow$ E)

Example 2: Setup

Graph Details

- Vertices: A, B, C, D $\quad (|V| = 4)$
- Source: A
- Edges (Weight): (A,B): 5, (B,C): 2, (C,D): -5, (D,B): -2
- $|E| = 4$

Bellman-Ford Parameters

- Passes needed: $|V| - 1 = 3$
- Check pass: 1 (Total 4 passes)
- Relaxation Order (Example): (A,B), (B,C), (C,D), (D,B)

Note: Cycle B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ B has weight $2 + (-5) + (-2) = -5$.

Example 2: Initialization Pass 1

Initialization (Pass 0)

- $d = \{A : 0, B : \infty, C : \infty, D : \infty\}$
- $\pi = \{A : \text{NIL}, B : \text{NIL}, C : \text{NIL}, D : \text{NIL}\}$

Pass 1 Relaxations

- Relax (A,B): $0 + 5 < \infty \implies d[B] = 5, \pi[B] = A$
- Relax (B,C): $5 + 2 < \infty \implies d[C] = 7, \pi[C] = B$
- Relax (C,D): $7 - 5 < \infty \implies d[D] = 2, \pi[D] = C$
- Relax (D,B): $2 - 2 = 0 < d[B] = 5 \implies d[B] = 0, \pi[B] = D$

End of Pass 1 State

- $d = \{A : 0, B : 0, C : 7, D : 2\}$
- $\pi = \{A : \text{NIL}, B : D, C : B, D : C\}$

Example 2: Pass 2

Pass 2 Relaxations (Using values from end of Pass 1)

- Relax (A,B): $0 + 5 \not< d[B] = 0$
- Relax (B,C): $0 + 2 = 2 < d[C] = 7 \implies d[C] = 2, \pi[C] = B$
- Relax (C,D): $2 - 5 = -3 < d[D] = 2 \implies d[D] = -3, \pi[D] = C$
- Relax (D,B): $-3 - 2 = -5 < d[B] = 0 \implies d[B] = -5, \pi[B] = D$

End of Pass 2 State

- $d = \{A : 0, B : -5, C : 2, D : -3\}$
- $\pi = \{A : \text{NIL}, B : D, C : B, D : C\}$

Example 2: Pass 3 ($|V| - 1$ Pass)

Pass 3 Relaxations (Using values from end of Pass 2)

- Relax (A,B): $0 + 5 \not< d[B] = -5$
- Relax (B,C): $-5 + 2 = -3 < d[C] = 2 \implies d[C] = -3, \pi[C] = B$
- Relax (C,D): $-3 - 5 = -8 < d[D] = -3 \implies d[D] = -8, \pi[D] = C$
- Relax (D,B): $-8 - 2 = -10 < d[B] = -5 \implies d[B] = -10, \pi[B] = D$

End of Pass 3 State

- $d = \{A : 0, B : -10, C : -3, D : -8\}$
- $\pi = \{A : \text{NIL}, B : D, C : B, D : C\}$

Example 2: Pass 4 (Negative Cycle Check)

Action: Relax all edges using final candidate distances from Pass 3.

- Check (A,B): $0 + 5 = 5 \not< d[B] = -10$
- Check (B,C): $d[B] + 2 = -10 + 2 = -8 < d[C] = -3$.
  - Update Detected! ($d[C]$ could be improved to -8).
- Check (C,D): (Further checks not strictly needed once one update is found)
- Check (D,B): (Further checks not strictly needed once one update is found)

Result

- An edge relaxation (B,C) resulted in an improvement during the $|V|$-th pass.
- This signals a Negative-weight cycle reachable from source A.

Example 2

Outcome

- The algorithm detected that distances could still be improved in Pass 4 (the check pass).
- This occurs because the negative cycle B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ B (weight -5) is reachable from A.
- Traversing the cycle allows path weights to decrease indefinitely.
- Therefore, shortest paths are not well-defined for nodes B, C, and D in this graph.
- Bellman-Ford successfully reports the existence of such a cycle.

Bellman-Ford: Runtime Complexity

- Initialization: $O(V)$
- Core Relaxation Loops: $V - 1$ passes, each relaxing E edges ($O(1)$/edge) $\implies O(VE)$ total.
- Negative Cycle Check: One additional pass relaxing E edges $\implies O(E)$.

Overall Runtime: The dominant term is $O(VE)$, so the total complexity is

$$\boldsymbol{O(VE)}$$

- Handles negative edge weights.
- Detects reachable negative cycles.
- Slower than Dijkstra's on graphs without negative edges.

Proof of Correctness for Bellman-Ford

- **Goal**: Show $d[v] = \text{dist}(s, v)$ for all $v \in V$.
- Proof has two main parts:
  1. $d[v] \geq \text{dist}(s, v)$: $d[v]$ never underestimates shortest path distance.
  2. $d[v] \leq \text{dist}(s, v)$: $d[v]$ eventually reaches the actual shortest path distance.

$d[v] \geq \text{dist}(s, v)$

- For each v, $d[v]$ is the total weight along a path from s to v.
- Since $\text{dist}(s, v)$ is defined as the minimum weight path, we have $d[v] \geq \text{dist}(s, v)$.

$d[v] \leq \text{dist}(s, v)$

- **Inductive Setup**:
  - Define $\text{dist}_i(s, v)$: minimum path weight from s to v using at most i edges.
  - Base case ($i = 0$): $d[s] = 0$ and $d[v] = +\infty$ for all $v \neq s$.
- **Inductive Step**: Assume for i-th iteration that $d[v] \leq \text{dist}_i(s, v)$.
  - For any $s \to v$ path P with $\leq i + 1$ edges, let u be the last vertex before v in P.
  - By induction, $d[u] \leq \text{dist}_i(s, u)$.
  - Update $d[v] = \min(d[v], d[u] + w_{uv})$, ensuring $d[v] \leq \text{dist}_{i+1}(s, v)$.
- After $n - 1$ iterations, $d[v] \leq \text{dist}(s, v)$.

Bellman-Ford Algorithm: Negative Weights

Negative Weight Handling:

- Algorithm works with negative weights if no negative cycles exist.
- Negative cycles can be detected if further updates occur in an n-th iteration.
- The algorithm still correctly return the shortest distance for vertices not connected to the negative cycle.
- If we can

Bellman-Ford Algorithm: Runtime Analysis

- **Each Iteration**: $O(m)$ time to scan all edges.
- **Total Time Complexity**: $O(n \cdot m)$, since algorithm iterates up to $n - 1$ times.
- **Summary**:
  - Suitable for graphs with negative weights.
  - Slower than Dijkstra's when all edge weights are positive.

Bellman-Ford: Sequential vs Parallel Updates

Sequential Edge-Relaxation Updates:

- Iteratively relaxes edges $|V| - 1$ times.
- For each edge (u, v):

$$d[v] = \min(d[v], d[u] + w(u, v))$$

- Updates are applied immediately, allowing new values to propagate within the same iteration.

Parallel Updates:

- Computes $d_i(v)$, the distance from s to v with at most i edges, for all vertices in parallel for each i (iteration):

$$d_i(v) = \min(d_{i-1}(s, v), d_{i-1}(u) + w(u, v))$$

- Uses values only from $i - 1$, ensuring no propagation within the same iteration.
- Maintains separate states for each i.

Bellman-Ford: Runtime Complexity

Core Loop:

- The algorithm performs $V - 1$ passes.
- In each pass, it iterates through and relaxes *all* E edges.
- Relaxation takes $O(1)$ time per edge.
- Cost of main loops: $(V - 1) \times O(E) = O(VE)$.

Negative Cycle Check:

- Requires one additional pass over all E edges.
- Cost: $O(E)$.

Initialization:

- Setting initial distances takes $O(V)$.

Overall Runtime:

- Total $= O(V) + O(VE) + O(E) = \boldsymbol{O(VE)}$.
- Slower than Dijkstra's for graphs with non-negative weights, especially sparse graphs.
- Main advantage: Ability to handle negative edge weights and detect negative cycles.

Cheapest Flights with K Stops

Problem Statement:

- **Input:**
  - n cities, list of flights (edges) as (source, destination, cost)
  - Starting city src, destination city dst, maximum stops K
- **Goal:** Find the minimum cost to travel from src to dst with at most K stops.

Cheapest Flights with K Stops

Problem Statement:

- **Input:**
  - n cities, list of flights (edges) as (source, destination, cost)
  - Starting city src, destination city dst, maximum stops K
- **Goal:** Find the minimum cost to travel from src to dst with at most K stops.

Solution: Bellman-Ford Approach

1. Initialize distance array dist with:

$$\text{dist[src]} = 0, \quad \text{dist[v]} = \infty \text{ for all other } v$$

2. For each iteration from 1 to $K + 1$ (representing stops):
   - Create a copy of dist as temp_dist to store updates for this iteration.
   - For each flight (u, v, cost), update:

$$\text{temp_dist[v]} = \min(\text{temp_dist[v]}, \text{dist[u]} + \text{cost})$$

   - Set dist = temp_dist at the end of the iteration.
3. Return dist[dst] if reachable, otherwise return -1.

Floyd-Warshall Algorithm Overview

- **Goal**: Compute shortest paths between all pairs of vertices.
- **Works with**: Positive and negative edge weights (no negative cycles).
- **Technique**: Dynamic programming via incremental path updates.
- **Output**: Distance matrix $\text{dist}[i][j]$ = shortest path length from i to j.
- **Complexity**:
  - Time: $O(n^3)$
  - Space: $O(n^2)$

Floyd-Warshall: Dynamic Programming Insight

Key Idea: $d_k[i][j]$ the shortest distance from i to j using $1, 2, \cdots k$ as intermediate vertices.

Recurrence:

$$d_k[i][j] = \min\left(d_{k-1}[i][j],\ d_{k-1}[i][k] + d_{k-1}[k][j]\right)$$

Explanation:

- Either the shortest path from i to j avoids k,
- Or it goes through k, in which case we combine:

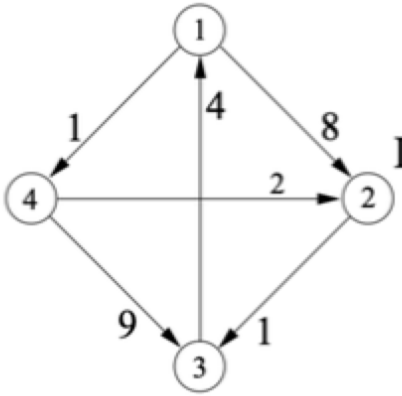
$$i \to k \quad + \quad k \to j$$

Floyd-Warshall: Pseudocode

1. Initialize distance matrix dist:
   - $\text{dist}[i][j] = w(i, j)$ if edge exists
   - $\text{dist}[i][j] = \infty$ otherwise
   - $\text{dist}[i][i] = 0$
2. Loop over all intermediate vertices k:
   - For all i, j:

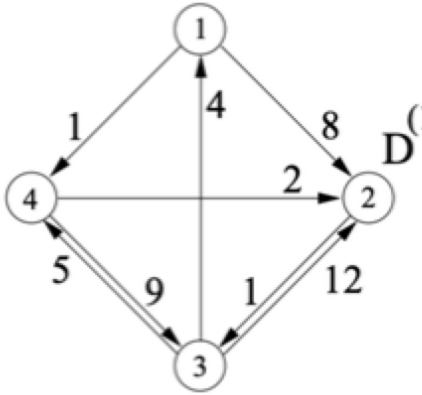
$$\text{dist}[i][j] \leftarrow \min(\text{dist}[i][j],\ \text{dist}[i][k] + \text{dist}[k][j])$$

Example

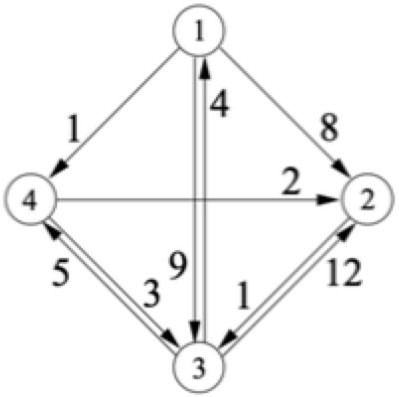


$$D^{(0)} = \begin{bmatrix} 0 & 8 & ? & 1 \\ ? & 0 & 1 & ? \\ 4 & ? & 0 & ? \\ ? & 2 & 9 & 0 \end{bmatrix}$$

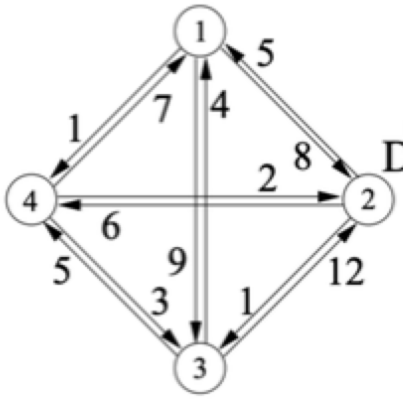
? = infinity



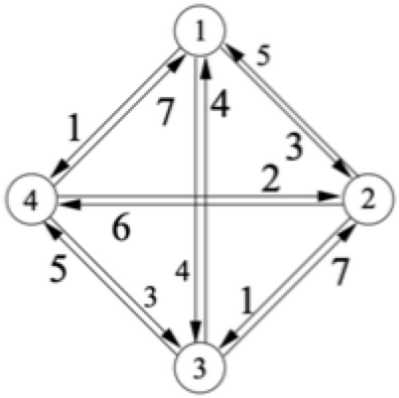
$$D^{(1)} = \begin{bmatrix} 0 & 8 & ? & 1 \\ ? & 0 & 1 & ? \\ 4 & 12 & 0 & 5 \\ ? & 2 & 9 & 0 \end{bmatrix}$$



$$D^{(2)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ ? & 0 & 1 & ? \\ 4 & 12 & 0 & 5 \\ ? & 2 & 3 & 0 \end{bmatrix}$$



$$D^{(3)} = \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$



$$D^{(4)} = \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix}$$

Sequential vs Parallel Updates

Sequential Updates (Floyd-Warshall):

- In-place updates: $\text{dist}[i][j]$ is updated immediately.
- Allows values to propagate within a single iteration over k.
- Memory efficient: only one distance matrix.

Parallel Updates (Layered DP):

- Compute $d_k[i][j]$ using only d_{k-1}:

$$d_k[i][j] = \min(d_{k-1}[i][j],\ d_{k-1}[i][k] + d_{k-1}[k][j])$$

- Each iteration uses a fresh matrix; updates do not affect same round.
- Easier to parallelize; more memory-intensive.

Negative Cycle Detection

- Floyd-Warshall can detect negative cycles.
- **Key Observation:** If $\text{dist}[i][i] < 0$ after the algorithm:
  - A negative-weight cycle exists that is reachable from i.
  - Shortest paths are undefined for vertices involved in such cycles.

Path Reconstruction

- Maintain a predecessor matrix $\text{pred}[i][j]$:
  - Initially: $\text{pred}[i][j] = i$ if edge (i, j) exists, else null.
  - When updating $\text{dist}[i][j]$ via k, set:

$$\text{pred}[i][j] = \text{pred}[k][j]$$

- **To reconstruct path from i to j:**
  - Follow the chain: $j \leftarrow \text{pred}[i][j] \leftarrow \cdots \leftarrow i$

Floyd-Warshall: Complexity Summary

- **Time Complexity:** $O(n^3)$
  - Triple nested loop over all vertex pairs and intermediates.
- **Space Complexity:** $O(n^2)$
  - Distance matrix for all pairs.
  - Optional: Predecessor matrix for path reconstruction.
- **Use Case:** Small or dense graphs; all-pairs shortest paths; graphs with negative weights.

City with the few close neighbours

Problem:

- Given n cities connected by weighted edges, find the city with the smallest number of neighbors within a given distance threshold.
- If multiple cities have the same number of reachable neighbors, return the city with the highest ID.

Approach:

1. Use Floyd-Warshall algorithm to compute all-pairs shortest paths between cities.
2. For each city i, count the number of cities j with distance$(i, j) \leq$ threshold.
3. Track the city with the smallest neighbor count; if tied, choose the city with the highest ID.